

# Forecasting the Term Structure of South African Government Bond Yields

A replication of Diebold and Li (2006), 1994 to 2026

## Software environment

```
In [1]: # Import sys, which gives information about the Python installation
import sys
# Import each library used in the notebook, so their versions can be printed
import pandas, numpy, statsmodels, matplotlib, scipy
# Print the Python version
print("Python:", sys.version.split()[0])
# Print the version of each library
print("pandas:", pandas.__version__)
print("numpy:", numpy.__version__)
print("statsmodels:", statsmodels.__version__)
print("matplotlib:", matplotlib.__version__)
print("scipy:", scipy.__version__)
```

```
Python: 3.13.9
pandas: 2.3.3
numpy: 2.3.5
statsmodels: 0.14.5
matplotlib: 3.10.6
scipy: 1.16.3
```

## Step 1: Load the SARB government bond yields

Monthly average yields for five maturity bands. Each band is assigned a maturity in months (band midpoint; 150 months assumed for "10 years and over").

```
In [2]: # Import pandas, the main Python library for working with tables of data
import pandas as pd
# Import numpy, the Python library for doing maths on lists of numbers
import numpy as np

# Location of the raw SARB bond yield file, relative to this notebook in the Code f
BOND_FILE = "../Data/Raw/SARB_govt_bond_yields_monthly.xlsx"

# Read the whole sheet with no headings, so SARB's metadata rows come in as ordinan
sheet = pd.read_excel(BOND_FILE, header=None)
# Find the row where the first column says "Date"; that row holds the real column h
header_row = sheet.index[sheet.iloc[:, 0] == "Date"][0]
# Use that row as the column headings of the table
sheet.columns = sheet.iloc[header_row]
# Remove the leftover label that pandas attaches to the column headings
sheet.columns.name = None
```

```

# Keep only the rows below the headings, which hold the actual yields
bonds = sheet.iloc[header_row + 1:].copy()

# Replace SARB's word "undefined" with a proper missing value
bonds = bonds.replace("undefined", np.nan)
# Turn text dates like "1990/01" into real dates set to the last day of each month
bonds["Date"] = pd.to_datetime(bonds["Date"].astype(str), format="%Y/%m") + pd.offsets.MonthEnd(1)
# Make the date the row label of the table
bonds = bonds.set_index("Date")
# Convert every yield column from text into numbers
bonds = bonds.astype(float)

# Dictionary linking each SARB code to a clear name showing its assigned maturity in months
BOND_NAMES = {
    "KBP2000M": "y_18m", # 0 to 3 years, midpoint 1.5 years = 18 months
    "KBP2001M": "y_48m", # 3 to 5 years, midpoint 4 years = 48 months
    "KBP2002M": "y_90m", # 5 to 10 years, midpoint 7.5 years = 90 months
    "KBP2003M": "y_150m", # 10 years and over, assumed 12.5 years = 150 months
    "KBP2049M": "y_300m", # 20 to 30 years, midpoint 25 years = 300 months (starts at 20 years)
}
# Rename the columns from SARB codes to the clear names
bonds = bonds.rename(columns=BOND_NAMES)
# Put the columns in order from shortest to longest maturity
bonds = bonds[list(BOND_NAMES.values())]

# Print the first five months of data
print(bonds.head())
# Print the last five months of data
print(bonds.tail())
# Print how many non-missing observations each maturity has
print(bonds.count())

```

|            | y_18m | y_48m | y_90m | y_150m | y_300m |
|------------|-------|-------|-------|--------|--------|
| Date       |       |       |       |        |        |
| 1990-01-31 | 15.46 | 15.45 | 16.64 | 15.49  | NaN    |
| 1990-02-28 | 15.72 | 15.33 | 16.64 | 15.61  | NaN    |
| 1990-03-31 | 15.60 | 15.39 | 16.64 | 15.66  | NaN    |
| 1990-04-30 | 15.64 | 15.52 | 16.64 | 16.16  | NaN    |
| 1990-05-31 | 15.59 | 15.52 | 16.64 | 16.17  | NaN    |
|            | y_18m | y_48m | y_90m | y_150m | y_300m |
| Date       |       |       |       |        |        |
| 2026-01-31 | 6.90  | 7.32  | 8.23  | 8.62   | 9.15   |
| 2026-02-28 | 6.83  | 7.13  | 7.97  | 8.26   | 8.69   |
| 2026-03-31 | 7.21  | 7.98  | 8.69  | 9.02   | 9.25   |
| 2026-04-30 | 7.50  | 7.93  | 8.60  | 8.92   | 9.27   |
| 2026-05-31 | 7.59  | 8.08  | 8.68  | 8.99   | 9.32   |
| y_18m      | 437   |       |       |        |        |
| y_48m      | 437   |       |       |        |        |
| y_90m      | 437   |       |       |        |        |
| y_150m     | 437   |       |       |        |        |
| y_300m     | 113   |       |       |        |        |
| dtype:     | int64 |       |       |        |        |

## Step 2: Load the SARB Treasury bill rates

Daily tender rates are averaged to monthly, to match the bond data. Treasury bills are quoted as discount rates, so each is converted to a yield (actual/365):

$$y = \frac{d}{1 - d \cdot \frac{n}{365}}$$

where  $d$  is the discount rate and  $n$  is the days to maturity.

```
In [3]: # Location of the raw SARB Treasury bill file, relative to this notebook in the Code
TBILL_FILE = "../Data/Raw/SARB_treasury_bills_daily.csv"

# Read the CSV file into a table
tbills_long = pd.read_csv(TBILL_FILE)
# Convert the Period column from text into real dates
tbills_long["Period"] = pd.to_datetime(tbills_long["Period"])
# Reshape the table so each Treasury bill gets its own column, with one row per date
tbills_daily = tbills_long.pivot_table(index="Period", columns="Series", values="Value")

# Label each date with its calendar month (for example, 1994-01)
month = tbills_daily.index.to_period("M")
# Average all the daily rates that fall within the same month
tbills_monthly = tbills_daily.groupby(month).mean()
# Turn the month labels back into dates on the last day of each month, matching the
tbills_monthly.index = tbills_monthly.index.to_timestamp() + pd.offsets.MonthEnd(0)
# Give the date index the same name as in the bond table
tbills_monthly.index.name = "Date"

# Dictionary linking each bill to its number of days to maturity
TBILL_DAYS = {"tbill_91d": 91, "tbill_182d": 182, "tbill_273d": 273, "tbill_364d": 364}
# Dictionary linking each bill to a clear name showing its maturity in months
TBILL_NAMES = {"tbill_91d": "y_3m", "tbill_182d": "y_6m", "tbill_273d": "y_9m", "tbill_364d": "y_12m"}

# Create an empty table with the same months, to hold the converted yields
tbill_yields = pd.DataFrame(index=tbills_monthly.index)
# Loop over each bill together with its number of days to maturity
for name, days in TBILL_DAYS.items():
    # Convert the discount rate from a percentage to a decimal (for example, 7.5 becomes 0.075)
    d = tbills_monthly[name] / 100
    # Apply the discount-to-yield formula, then convert back to a percentage
    tbill_yields[TBILL_NAMES[name]] = 100 * d / (1 - d * days / 365)

# Print the first five months of converted yields
print(tbill_yields.head())
# Print the last five months of converted yields
print(tbill_yields.tail())
# Print how many non-missing months each bill has
print(tbill_yields.count())
# Print the average gap between the yield and the original discount rate, in percentage points
print((tbill_yields["y_3m"] - tbills_monthly["tbill_91d"]).mean())
```

|            | y_3m      | y_6m | y_9m | y_12m |
|------------|-----------|------|------|-------|
| Date       |           |      |      |       |
| 1990-01-31 | 18.845735 | NaN  | NaN  | NaN   |
| 1990-02-28 | 18.850759 | NaN  | NaN  | NaN   |
| 1990-03-31 | 18.803922 | NaN  | NaN  | NaN   |
| 1990-04-30 | 18.829293 | NaN  | NaN  | NaN   |
| 1990-05-31 | 18.849389 | NaN  | NaN  | NaN   |

|            | y_3m     | y_6m     | y_9m     | y_12m    |
|------------|----------|----------|----------|----------|
| Date       |          |          |          |          |
| 2026-05-31 | 7.031130 | 7.474776 | 7.763169 | 7.787739 |
| 2026-06-30 | 7.239296 | 7.537301 | 7.755173 | 7.810058 |
| 2026-07-31 | 7.110373 | 7.477004 | 7.646239 | 7.731745 |
| 2026-08-31 | 6.949352 | 7.412723 | 7.664002 | 7.748848 |
| 2026-09-30 | 6.953797 | 7.558021 | 7.767684 | 7.871798 |

y\_3m 441  
 y\_6m 297  
 y\_9m 236  
 y\_12m 196  
 dtype: int64  
 0.2573717177245484

## Step 3: Combine and set the sample

Treasury bill and bond yields are joined and the sample is set to January 1994 to May 2026.

- **Main specification:** five maturities available in every month (3, 18, 48, 90, 150 months), mirroring Diebold and Li's fixed maturity set.
- **Robustness:** all available maturities (up to nine).

```
In [4]: # Join the Treasury bill yields and the bond yields side by side, keeping only mont
yields_all = tbill_yields.join(bonds, how="inner")

# List of every yield column, ordered from shortest to longest maturity
ALL_COLS = ["y_3m", "y_6m", "y_9m", "y_12m", "y_18m", "y_48m", "y_90m", "y_150m", "
# Reorder the combined table's columns to follow that list
yields_all = yields_all[ALL_COLS]

# First month of the sample, after South Africa's re-entry into global capital mark
SAMPLE_START = "1994-01-31"
# Last month of the sample, set by the latest available SARB bond yields
SAMPLE_END = "2026-05-31"
# Keep only the months inside the sample period
yields_all = yields_all.loc[SAMPLE_START:SAMPLE_END]

# Dictionary linking each yield column to its maturity in months
MATURITY = {"y_3m": 3, "y_6m": 6, "y_9m": 9, "y_12m": 12, "y_18m": 18,
            "y_48m": 48, "y_90m": 90, "y_150m": 150, "y_300m": 300}

# Columns used in the main specification: the five maturities available in every mo
MAIN_COLS = ["y_3m", "y_18m", "y_48m", "y_90m", "y_150m"]
# Table of yields for the main specification
yields = yields_all[MAIN_COLS]
# Maturities (in months) of the main specification, stored as an array of numbers
tau = np.array([MATURITY[c] for c in MAIN_COLS])
```

```

# Save the cleaned, combined dataset (all maturities) to the Output folder
yields_all.to_csv("../Output/sa_yields_monthly_clean.csv")

# Print the first and last month of the sample
print("Sample:", yields.index[0].date(), "to", yields.index[-1].date())
# Print the number of months (rows) and maturities (columns) in the main specification
print("Main specification shape:", yields.shape)
# Print the total number of missing values in the main specification (should be 0)
print("Missing values in main specification:", yields.isna().sum().sum())
# Print the maturities used in the main specification
print("Maturities (months):", tau)
# Print how many months of data each maturity has in the full table
print(yields_all.count())

```

```

Sample: 1994-01-31 to 2026-05-31
Main specification shape: (389, 5)
Missing values in main specification: 0
Maturities (months): [ 3  18  48  90 150]
y_3m      389
y_6m      293
y_9m      232
y_12m     192
y_18m     389
y_48m     389
y_90m     389
y_150m    389
y_300m    113
dtype: int64

```

## Data check: zero yields

The SARB 3 to 5 year series (KBP2001M) contains values of exactly zero, which are not valid government bond yields and most likely reflect missing observations recorded as zero in the source data. The cells below identify the affected months before any correction is applied.

```

In [5]: # Find every month where any yield in the main specification is zero or negative
bad_months = yields[(yields <= 0).any(axis=1)]
# Print those months, so we can see which dates and maturities are affected
print(bad_months)

# Loop over each problem month
for date in bad_months.index:
    # Find the position (row number) of this month in the yields table
    pos = yields.index.get_loc(date)
    # Print the two months before and after, to see the values around the problem
    print(yields.iloc[pos - 2 : pos + 3])

```

|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
|------------|----------|-------|-------|-------|--------|
| Date       |          |       |       |       |        |
| 2023-03-31 | 7.219936 | 8.43  | 0.0   | 10.03 | 11.19  |
| 2023-04-30 | 7.870281 | 8.62  | 0.0   | 9.97  | 11.26  |
| 2023-05-31 | 8.207771 | 9.42  | 0.0   | 10.78 | 11.96  |
| 2023-06-30 | 8.669120 | 9.44  | 0.0   | 10.82 | 11.95  |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |
| 2024-10-31 | 8.166883 | 8.25  | 0.0   | 9.23  | 10.46  |
| 2024-11-30 | 8.082510 | 8.28  | 0.0   | 9.15  | 10.39  |
| 2024-12-31 | 7.958523 | 8.18  | 0.0   | 8.96  | 10.23  |
| 2025-01-31 | 7.898914 | 8.29  | 0.0   | 9.10  | 10.44  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-01-31 | 6.574109 | 7.54  | 8.25  | 9.79  | 10.95  |
| 2023-02-28 | 6.691295 | 7.58  | 8.41  | 9.89  | 11.05  |
| 2023-03-31 | 7.219936 | 8.43  | 0.00  | 10.03 | 11.19  |
| 2023-04-30 | 7.870281 | 8.62  | 0.00  | 9.97  | 11.26  |
| 2023-05-31 | 8.207771 | 9.42  | 0.00  | 10.78 | 11.96  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-02-28 | 6.691295 | 7.58  | 8.41  | 9.89  | 11.05  |
| 2023-03-31 | 7.219936 | 8.43  | 0.00  | 10.03 | 11.19  |
| 2023-04-30 | 7.870281 | 8.62  | 0.00  | 9.97  | 11.26  |
| 2023-05-31 | 8.207771 | 9.42  | 0.00  | 10.78 | 11.96  |
| 2023-06-30 | 8.669120 | 9.44  | 0.00  | 10.82 | 11.95  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-03-31 | 7.219936 | 8.43  | 0.0   | 10.03 | 11.19  |
| 2023-04-30 | 7.870281 | 8.62  | 0.0   | 9.97  | 11.26  |
| 2023-05-31 | 8.207771 | 9.42  | 0.0   | 10.78 | 11.96  |
| 2023-06-30 | 8.669120 | 9.44  | 0.0   | 10.82 | 11.95  |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-04-30 | 7.870281 | 8.62  | 0.0   | 9.97  | 11.26  |
| 2023-05-31 | 8.207771 | 9.42  | 0.0   | 10.78 | 11.96  |
| 2023-06-30 | 8.669120 | 9.44  | 0.0   | 10.82 | 11.95  |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-05-31 | 8.207771 | 9.42  | 0.0   | 10.78 | 11.96  |

|            |          |       |       |       |        |
|------------|----------|-------|-------|-------|--------|
| 2023-06-30 | 8.669120 | 9.44  | 0.0   | 10.82 | 11.95  |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-06-30 | 8.669120 | 9.44  | 0.0   | 10.82 | 11.95  |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-07-31 | 8.754621 | 9.08  | 0.0   | 10.41 | 11.73  |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-08-31 | 8.618568 | 8.99  | 0.0   | 10.30 | 11.73  |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-09-30 | 8.500930 | 9.15  | 0.0   | 10.52 | 12.07  |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-10-31 | 8.738123 | 9.41  | 0.0   | 10.81 | 12.36  |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-11-30 | 8.698264 | 8.93  | 0.0   | 10.21 | 11.80  |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2023-12-31 | 8.628950 | 8.76  | 0.0   | 9.88  | 11.53  |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-01-31 | 8.659835 | 8.70  | 0.0   | 9.74  | 11.42  |

|            |          |       |       |       |        |
|------------|----------|-------|-------|-------|--------|
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-02-29 | 8.642284 | 8.85  | 0.0   | 10.01 | 11.63  |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-03-31 | 8.627455 | 9.07  | 0.0   | 10.29 | 11.91  |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-04-30 | 8.612470 | 9.35  | 0.0   | 10.66 | 12.27  |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-05-31 | 8.715346 | 9.17  | 0.0   | 10.47 | 12.05  |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-06-30 | 8.718642 | 8.89  | 0.0   | 10.15 | 11.66  |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |
| 2024-10-31 | 8.166883 | 8.25  | 0.0   | 9.23  | 10.46  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-07-31 | 8.631395 | 8.51  | 0.0   | 9.63  | 11.02  |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |
| 2024-10-31 | 8.166883 | 8.25  | 0.0   | 9.23  | 10.46  |
| 2024-11-30 | 8.082510 | 8.28  | 0.0   | 9.15  | 10.39  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-08-31 | 8.392479 | 8.23  | 0.0   | 9.25  | 10.73  |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |
| 2024-10-31 | 8.166883 | 8.25  | 0.0   | 9.23  | 10.46  |
| 2024-11-30 | 8.082510 | 8.28  | 0.0   | 9.15  | 10.39  |
| 2024-12-31 | 7.958523 | 8.18  | 0.0   | 8.96  | 10.23  |
|            | y_3m     | y_18m | y_48m | y_90m | y_150m |
| Date       |          |       |       |       |        |
| 2024-09-30 | 8.262260 | 8.01  | 0.0   | 8.93  | 10.32  |

| Date       | y_3m     | y_18m | y_48m | y_90m | y_150m |
|------------|----------|-------|-------|-------|--------|
| 2024-10-31 | 8.166883 | 8.25  | 0.0   | 9.23  | 10.46  |
| 2024-11-30 | 8.082510 | 8.28  | 0.0   | 9.15  | 10.39  |
| 2024-12-31 | 7.958523 | 8.18  | 0.0   | 8.96  | 10.23  |
| 2025-01-31 | 7.898914 | 8.29  | 0.0   | 9.10  | 10.44  |
| 2025-02-28 | 7.554661 | 8.28  | 9.11  | 10.50 | 10.92  |
| 2025-03-31 | 7.497577 | 8.26  | 9.09  | 10.57 | 10.99  |

```
In [6]: # Collect the dates of every month where the 3 to 5 year yield is zero or negative
zero_48 = yields.index[yields["y_48m"] <= 0]
# Print how many months are affected
print("Months with zero y_48m:", len(zero_48))
# Print the first and last affected month
print("First:", zero_48.min().date(), " Last:", zero_48.max().date())
# Print the last month that still has a valid (positive) 3 to 5 year yield
print("Last valid y_48m:", yields.index[yields["y_48m"] > 0].max().date())
# Print how many zero or negative values each maturity has, to confirm only y_48m i
print((yields <= 0).sum())
```

```
Months with zero y_48m: 23
First: 2023-03-31 Last: 2025-01-31
Last valid y_48m: 2026-05-31
y_3m      0
y_18m     0
y_48m     23
y_90m     0
y_150m    0
dtype: int64
```

## Data correction and final sample

The zeros in the 3 to 5 year series cover 23 consecutive months (March 2023 to January 2025), after which the series resumes. The zeros are set to missing. The gap is too long to interpolate reliably, and the fixed-maturity design requires all five maturities in every month, so the **main specification ends in February 2023**. The full sample to May 2026 is retained for the robustness analysis, which fits each month using all available maturities.

```
In [7]: # Replace the zero 3 to 5 year yields with missing values, since zero is not a valid
yields_all.loc[yields_all["y_48m"] <= 0, "y_48m"] = np.nan
# Save the corrected full dataset (all maturities, to May 2026) for the robustness
yields_all.to_csv("../Output/sa_yields_monthly_clean.csv")

# End of the main sample: the last month before the first zero in the 3 to 5 year s
```

```

MAIN_END = zero_48.min() - pd.offsets.MonthEnd(1)
# Main specification: the five fixed maturities, from January 1994 to the end of th
yields = yields_all.loc[:MAIN_END, MAIN_COLS]

# Print the first and last month of the main sample
print("Main sample:", yields.index[0].date(), "to", yields.index[-1].date())
# Print the number of months (rows) and maturities (columns) in the main specificat
print("Main specification shape:", yields.shape)
# Print the total number of missing values in the main specification (should be 0)
print("Missing values in main specification:", yields.isna().sum().sum())
# Print the smallest yield in each column, to confirm no zeros remain
print(yields.min())

```

```

Main sample: 1994-01-31 to 2023-02-28
Main specification shape: (350, 5)
Missing values in main specification: 0
y_3m      2.999261
y_18m     4.150000
y_48m     5.280000
y_90m     6.090000
y_150m    6.960000
dtype: float64

```

## Step 4: Descriptive statistics (Table 1)

Summary statistics and sample autocorrelations at lags 1, 12 and 30 months, following Diebold and Li's Table 1. Empirical factor proxies:

- **Level** = 150-month yield
- **Slope** = 150-month yield minus 3-month yield
- **Curvature** =  $2 \times$  18-month yield minus 3-month yield minus 150-month yield

```

In [8]: # Import the acf function from statsmodels, which calculates sample autocorrelation
from statsmodels.tsa.stattools import acf

# Tell pandas to print wide tables on one line instead of wrapping them
pd.set_option("display.width", 200)

# Empirical level: the longest maturity in the main specification (150 months)
level_emp = yields["y_150m"]
# Empirical slope: the long yield minus the short yield
slope_emp = yields["y_150m"] - yields["y_3m"]
# Empirical curvature: twice the medium yield minus the short and long yields
curv_emp = 2 * yields["y_18m"] - yields["y_3m"] - yields["y_150m"]

# Copy the yields table so the original stays unchanged
desc_data = yields.copy()
# Relabel the 150-month column to show it also serves as the Level proxy
desc_data = desc_data.rename(columns={"y_150m": "y_150m (level)"})
# Add the slope proxy as a new column
desc_data["Slope"] = slope_emp
# Add the curvature proxy as a new column
desc_data["Curvature"] = curv_emp

```

```

# Empty list that will hold one row of statistics for each series
rows = []
# Loop over every column in the table
for col in desc_data.columns:
    # Pick out the data for this one series
    s = desc_data[col]
    # Calculate the sample autocorrelations from lag 0 up to lag 30
    r = acf(s, nlags=30)
    # Store the summary statistics for this series as one row
    rows.append({
        "Series": col,          # name of the series
        "Mean": s.mean(),      # average value
        "Std. Dev.": s.std(),   # standard deviation (how much it moves around)
        "Minimum": s.min(),    # lowest value
        "Maximum": s.max(),    # highest value
        "rho(1)": r[1],        # autocorrelation with the value 1 month earlier
        "rho(12)": r[12],      # autocorrelation with the value 12 months earlier
        "rho(30)": r[30],      # autocorrelation with the value 30 months earlier
    })

# Turn the list of rows into a table, using the series name as the row label
table1 = pd.DataFrame(rows).set_index("Series")

# Print a title for the table
print("Table 1: Descriptive statistics, South African yield curves, 1994:01 to 2023:02")
# Print the table rounded to 3 decimal places
print(table1.round(3))

# Build a table holding the three empirical factor proxies side by side
proxies = pd.DataFrame({"Level": level_emp, "Slope": slope_emp, "Curvature": curv_emp})
# Print a title for the correlation table
print("\nCorrelations between empirical level, slope and curvature")
# Print the correlation matrix rounded to 3 decimal places
print(proxies.corr().round(3))

```

Table 1: Descriptive statistics, South African yield curves, 1994:01 to 2023:02

|                | Mean   | Std. Dev. | Minimum | Maximum | rho(1) | rho(12) | rho(30) |
|----------------|--------|-----------|---------|---------|--------|---------|---------|
| Series         |        |           |         |         |        |         |         |
| y_3m           | 8.888  | 3.701     | 2.999   | 22.873  | 0.991  | 0.787   | 0.498   |
| y_18m          | 9.151  | 3.385     | 4.150   | 19.790  | 0.989  | 0.824   | 0.590   |
| y_48m          | 9.760  | 3.126     | 5.280   | 19.760  | 0.989  | 0.835   | 0.654   |
| y_90m          | 10.161 | 2.933     | 6.090   | 18.380  | 0.990  | 0.851   | 0.686   |
| y_150m (level) | 10.520 | 2.785     | 6.960   | 18.300  | 0.990  | 0.847   | 0.670   |
| Slope          | 1.632  | 2.250     | -4.573  | 6.869   | 0.978  | 0.492   | -0.003  |
| Curvature      | -1.106 | 1.561     | -5.284  | 2.871   | 0.932  | 0.472   | 0.128   |

Correlations between empirical level, slope and curvature

|           | Level  | Slope | Curvature |
|-----------|--------|-------|-----------|
| Level     | 1.000  | -0.07 | 0.157     |
| Slope     | -0.070 | 1.00  | -0.400    |
| Curvature | 0.157  | -0.40 | 1.000     |

## Step 5: The yield curve over time (Figures 1 and 2)

Figure 1 shows the full yield surface across maturity and time (Diebold and Li's Figure 2). Figure 2 shows the median yield curve with the 25th and 75th percentiles at each maturity (their Figure 3).

```
In [9]: # Import matplotlib's plotting module, which draws charts
import matplotlib.pyplot as plt

# Use Calibri as the font for all figures, to match the report
plt.rcParams["font.family"] = "Calibri"
# Set the default font size for all figures
plt.rcParams["font.size"] = 11

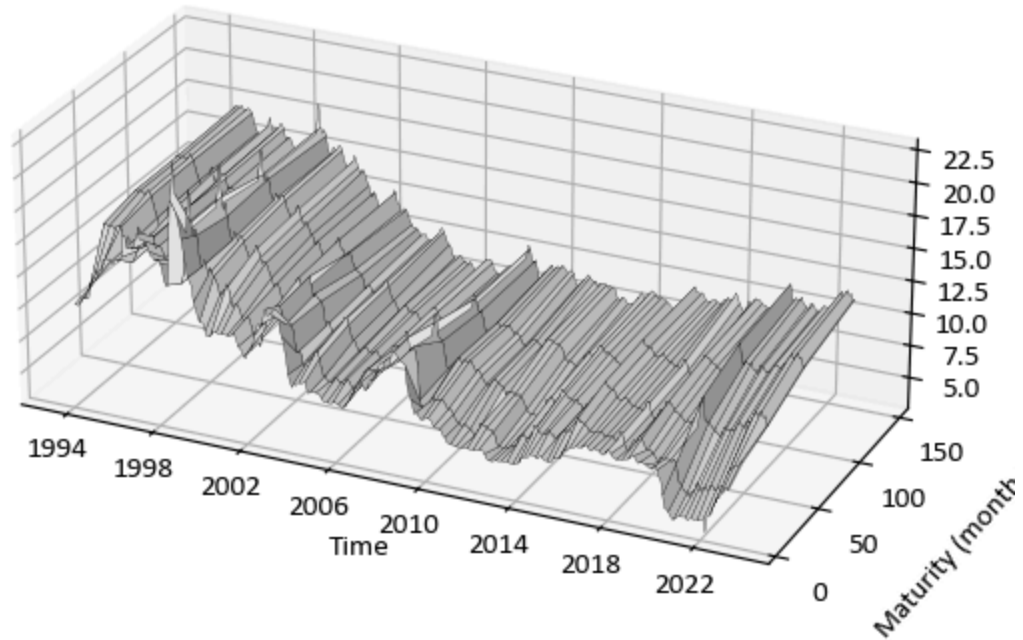
# Build two grids: one holding the month number (time) and one holding the maturity
T, M = np.meshgrid(np.arange(len(yields)), tau)

# Create an empty figure, 11 by 7 inches
fig = plt.figure(figsize=(11, 7))
# Add a set of 3D axes to the figure
ax = fig.add_subplot(111, projection="3d")
# Draw the yield surface: white fill with thin black grid lines (black and white fo
ax.plot_surface(T, M, yields.values.T, color="white", edgecolor="black",
                linewidth=0.2, rstride=1, cstride=3, shade=True)

# Find the row positions of January in every fourth year, to use as labels on the t
pos = [i for i, d in enumerate(yields.index) if d.month == 1 and d.year % 4 == 2]
# Place tick marks at those positions on the time axis
ax.set_xticks(pos)
# Label those tick marks with the year
ax.set_xticklabels([yields.index[i].year for i in pos])
# Place tick marks on the maturity axis every 50 months
ax.set_yticks([0, 50, 100, 150])

# Label the time axis
ax.set_xlabel("Time", labelpad=10)
# Label the maturity axis
ax.set_ylabel("Maturity (months)", labelpad=10)
# Label the yield axis
ax.set_zlabel("Yield (percent)", labelpad=8)
# Stretch the time axis so the 29 years are easy to read, and zoom out slightly so
ax.set_box_aspect((2.2, 1, 0.8), zoom=0.85)
# Set the viewing angle: 25 degrees above, rotated so time runs along the front
ax.view_init(elev=25, azimuth=-65)

# Save the figure to the Output folder at print quality (300 dots per inch)
fig.savefig("../Output/fig1_yield_surface.png", dpi=300, bbox_inches="tight", pad_i
# Show the figure in the notebook
plt.show()
```



```
In [10]: # Median yield at each maturity across all months
med = yields.median()
# 25th percentile of yields at each maturity
q25 = yields.quantile(0.25)
# 75th percentile of yields at each maturity
q75 = yields.quantile(0.75)

# Put the three curves in one table, with maturity in months as the row label
curve_table = pd.DataFrame({"25th pct": q25.values, "Median": med.values, "75th pct": q75.values})
# Name the row label so the printed table is clear
curve_table.index.name = "Maturity (months)"
# Print the values behind Figure 2, rounded to 3 decimal places
print(curve_table.round(3))

# Create a figure with one set of axes, 8 by 5 inches
fig, ax = plt.subplots(figsize=(8, 5))
# Draw the median curve as a thick solid black line with circle markers
ax.plot(tau, med.values, color="black", linewidth=2, marker="o", label="Median")
# Draw the 75th percentile as a thin dashed line with upward triangles
ax.plot(tau, q75.values, color="black", linewidth=1, linestyle="--", marker="^", label="75th pct")
# Draw the 25th percentile as a thin dotted line with downward triangles
ax.plot(tau, q25.values, color="black", linewidth=1, linestyle=":", marker="v", label="25th pct")
```

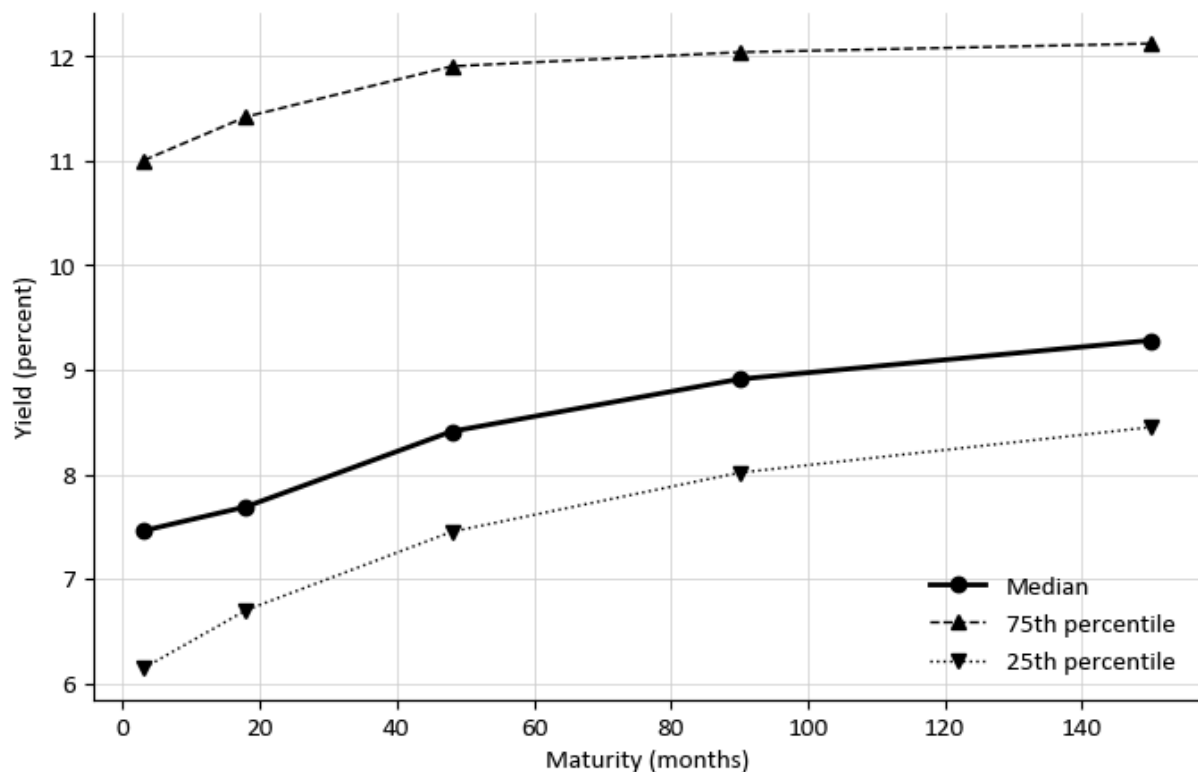
```

# Label the horizontal axis
ax.set_xlabel("Maturity (months)")
# Label the vertical axis
ax.set_ylabel("Yield (percent)")
# Add a Legend without a box around it
ax.legend(frameon=False)
# Add light grey grid lines to help read values
ax.grid(color="lightgrey", linewidth=0.5)
# Remove the top and right borders for a cleaner look
ax.spines[["top", "right"]].set_visible(False)

# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig2_median_curve.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()

```

|                   | 25th pct | Median | 75th pct |
|-------------------|----------|--------|----------|
| Maturity (months) |          |        |          |
| 3                 | 6.145    | 7.46   | 11.002   |
| 18                | 6.700    | 7.69   | 11.420   |
| 48                | 7.450    | 8.41   | 11.902   |
| 90                | 8.015    | 8.91   | 12.038   |
| 150               | 8.450    | 9.28   | 12.120   |



## Step 6: Nelson-Siegel factor estimation

$$y_t(\tau) = \beta_{1t} + \beta_{2t} \left( \frac{1 - e^{-\lambda\tau}}{\lambda\tau} \right) + \beta_{3t} \left( \frac{1 - e^{-\lambda\tau}}{\lambda\tau} - e^{-\lambda\tau} \right) + \varepsilon_t(\tau)$$

Following Diebold and Li,  $\lambda$  is fixed at 0.0609. They state this places the curvature peak at 30 months; the exact peak is 29.4 months (an exact 30-month peak would require  $\lambda \approx 0.0598$ ), a negligible difference. With  $\lambda$  fixed, the loadings are known, and  $\beta_{1t}, \beta_{2t}, \beta_{3t}$  are estimated by OLS for each month.

```
In [11]: # Fixed decay parameter, the same value used by Diebold and Li
LAMBDA = 0.0609

# Define a function that returns the three Nelson-Siegel loadings for a list of mat
def ns_loadings(maturities, lam):
    # Convert the maturities into a numpy array of decimal numbers
    m = np.asarray(maturities, dtype=float)
    # Slope Loading: starts near 1 at short maturities and decays towards 0
    slope_load = (1 - np.exp(-lam * m)) / (lam * m)
    # Curvature Loading: starts near 0, rises to a hump, then decays towards 0
    curv_load = slope_load - np.exp(-lam * m)
    # Return a table with three columns: Level Loading (all ones), slope Loading an
    return np.column_stack([np.ones_like(m), slope_load, curv_load])

# Fine grid of maturities from 1 to 360 months, in steps of 0.1 months
grid = np.arange(1, 360.01, 0.1)
# Calculate the three loadings at every point on the grid
L_grid = ns_loadings(grid, LAMBDA)
# Print the maturity at which the curvature loading reaches its highest point (shou
print("Curvature loading peaks at", round(grid[np.argmax(L_grid[:, 2])], 1), "month

# Calculate the loadings at the five maturities used in the main specification
loadings_table = pd.DataFrame(ns_loadings(tau, LAMBDA), index=tau, columns=["Level"
# Name the row label so the printed table is clear
loadings_table.index.name = "Maturity (months)"
# Print the loadings at the five maturities, rounded to 4 decimal places
print(loadings_table.round(4))

# Keep only the part of the grid up to 150 months, the longest maturity in the main
show = grid <= 150
# Create a figure with one set of axes, 8 by 5 inches
fig, ax = plt.subplots(figsize=(8, 5))
# Draw the Level Loading as a dashed line
ax.plot(grid[show], L_grid[show, 0], color="black", linewidth=1.5, linestyle="--",
# Draw the slope Loading as a solid line
ax.plot(grid[show], L_grid[show, 1], color="black", linewidth=1.5, linestyle="-", 1
# Draw the curvature Loading as a dotted line
ax.plot(grid[show], L_grid[show, 2], color="black", linewidth=1.5, linestyle=":", 1
# Mark the five maturities actually observed with small circles on the slope and cu
ax.plot(tau, loadings_table["Slope"], "o", color="black", markersize=5)
# Mark the five observed maturities on the curvature curve with hollow circles
ax.plot(tau, loadings_table["Curvature"], "o", color="black", markerfacecolor="whit
# Label the horizontal axis
ax.set_xlabel("Maturity (months)")
# Label the vertical axis
ax.set_ylabel("Loading")
# Set the vertical axis to run from 0 to 1.1
ax.set_ylim(0, 1.1)
# Add a Legend without a box
```

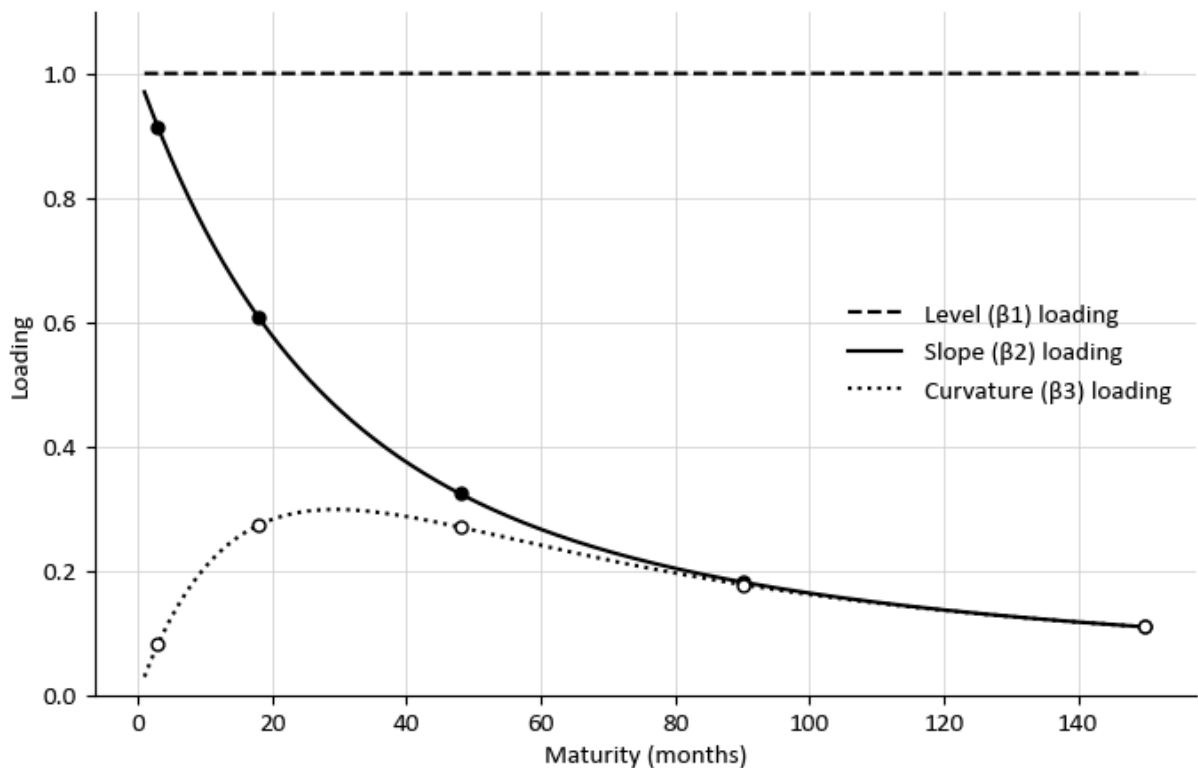
```

ax.legend(frameon=False)
# Add light grey grid lines
ax.grid(color="lightgrey", linewidth=0.5)
# Remove the top and right borders
ax.spines[["top", "right"]].set_visible(False)
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig3_factor_loadings.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()

```

Curvature loading peaks at 29.4 months

|                   | Level | Slope  | Curvature |
|-------------------|-------|--------|-----------|
| Maturity (months) |       |        |           |
| 3                 | 1.0   | 0.9140 | 0.0810    |
| 18                | 1.0   | 0.6074 | 0.2733    |
| 48                | 1.0   | 0.3237 | 0.2699    |
| 90                | 1.0   | 0.1817 | 0.1775    |
| 150               | 1.0   | 0.1095 | 0.1093    |



```

In [12]: # Loadings matrix for the five maturities: 5 rows (maturities) by 3 columns (level,
X = ns_loadings(tau, LAMBDA)

# Run OLS for every month at once: each column of yields.values.T is one month's cu
# and lstsq finds the three betas that best fit each column
coef = np.linalg.lstsq(X, yields.values.T, rcond=None)[0]
# Put the estimates in a table with one row per month and one column per factor
factors = pd.DataFrame(coef.T, index=yields.index, columns=["beta1", "beta2", "beta3"])

# Fitted yields: rebuild each month's curve from its three estimated betas
fitted = pd.DataFrame(factors.values @ X.T, index=yields.index, columns=yields.columns)
# Residuals (pricing errors): actual yield minus fitted yield
resid = yields - fitted

```

```
# Save the estimated factors to the Output folder
factors.to_csv("../Output/ns_factors_main.csv")

# Print the first five months of estimated factors
print(factors.head())
# Print summary statistics of the estimated factors, rounded to 3 decimal places
print(factors.describe().round(3))

# Print the correlation between the estimated level factor and the empirical level
print("corr(beta1, level): ", round(factors["beta1"].corr(level_emp), 3))
# Print the correlation between the estimated slope factor and the empirical slope
print("corr(beta2, slope): ", round(factors["beta2"].corr(slope_emp), 3))
# Print the correlation between the estimated curvature factor and the empirical cu
print("corr(beta3, curvature):", round(factors["beta3"].corr(curv_emp), 3))
```

|                         | beta1     | beta2     | beta3     |
|-------------------------|-----------|-----------|-----------|
| Date                    |           |           |           |
| 1994-01-31              | 12.674710 | -2.352107 | -1.416751 |
| 1994-02-28              | 13.135976 | -2.925845 | -0.294819 |
| 1994-03-31              | 13.416621 | -3.226346 | -0.708267 |
| 1994-04-30              | 13.688660 | -3.353015 | -0.621198 |
| 1994-05-31              | 14.127654 | -3.295250 | -1.488014 |
|                         | beta1     | beta2     | beta3     |
| count                   | 350.000   | 350.000   | 350.000   |
| mean                    | 10.925    | -2.057    | -1.923    |
| std                     | 2.777     | 2.802     | 3.410     |
| min                     | 7.027     | -9.044    | -10.544   |
| 25%                     | 8.815     | -3.325    | -4.381    |
| 50%                     | 9.820     | -2.214    | -2.115    |
| 75%                     | 12.834    | -0.590    | 0.255     |
| max                     | 18.260    | 5.126     | 8.872     |
| corr(beta1, level):     |           | 0.979     |           |
| corr(beta2, slope):     |           | -0.995    |           |
| corr(beta3, curvature): |           | 0.941     |           |

## Step 7: In-sample fit

Residual (pricing error) statistics by maturity, following Diebold and Li's Table 2; the average fitted curve against the average actual curve (their Figure 4); and fitted curves on selected dates (their Figure 5).

```
In [13]: # Empty list that will hold one row of residual statistics for each maturity
rows = []
# Loop over each maturity column in the residuals table
for col in resid.columns:
    # Pick out the residuals for this maturity
    e = resid[col]
    # Calculate the residual autocorrelations from lag 0 up to lag 30
    r = acf(e, nlags=30)
    # Store the statistics for this maturity as one row
    rows.append({
        "Maturity": MATURITY[col],          # maturity in months
        "Mean": e.mean(),                   # average residual (bias)
        "Std. Dev.": e.std(),               # standard deviation of the residuals
```

```

    "Minimum": e.min(),                # largest negative error
    "Maximum": e.max(),                # largest positive error
    "MAE": e.abs().mean(),             # mean absolute error
    "RMSE": np.sqrt((e ** 2).mean()), # root mean squared error
    "rho(1)": r[1],                   # residual autocorrelation at lag 1
    "rho(12)": r[12],                 # residual autocorrelation at lag 12
    "rho(30)": r[30],                 # residual autocorrelation at lag 30
})

# Turn the rows into a table, with maturity as the row label
table2 = pd.DataFrame(rows).set_index("Maturity")
# Print a title for the table
print("Table 2: Descriptive statistics, yield curve residuals, 1994:01 to 2023:02 (
# Print the table rounded to 3 decimal places
print(table2.round(3))

```

Table 2: Descriptive statistics, yield curve residuals, 1994:01 to 2023:02 (percentage points)

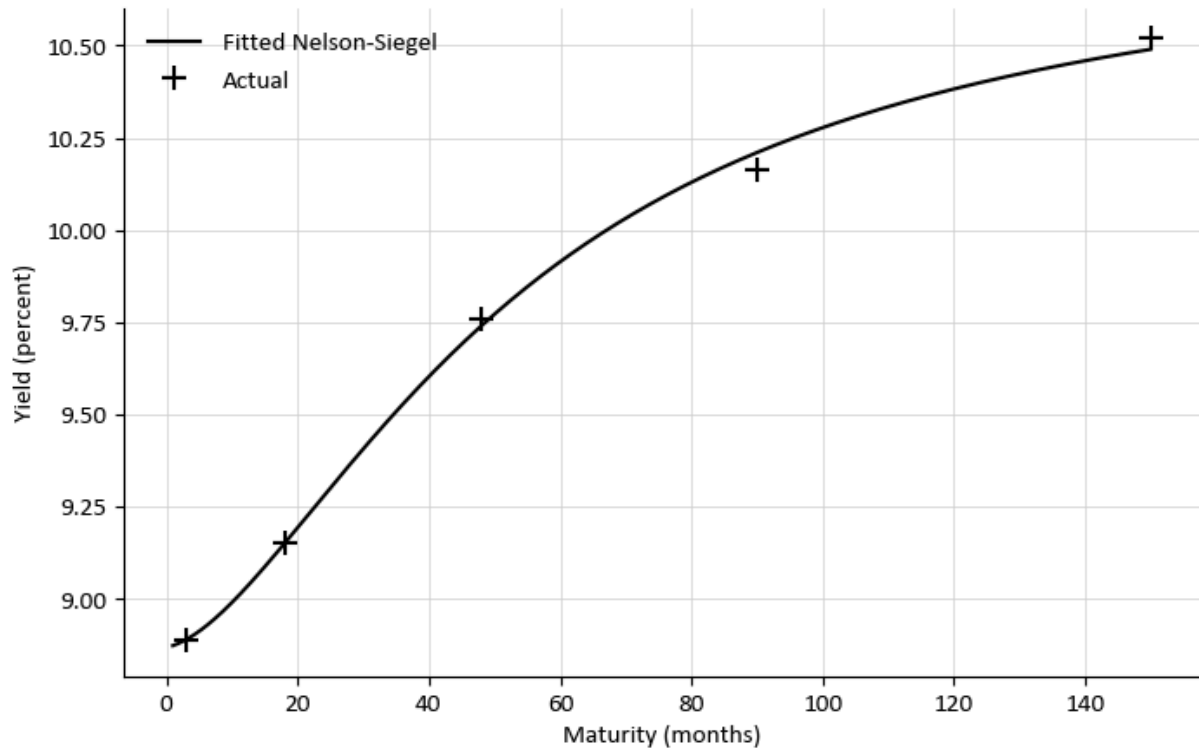
|          | Mean   | Std. Dev. | Minimum | Maximum | MAE   | RMSE  | rho(1) | rho(12) | rho(30) |
|----------|--------|-----------|---------|---------|-------|-------|--------|---------|---------|
| Maturity |        |           |         |         |       |       |        |         |         |
| 3        | -0.001 | 0.064     | -0.143  | 0.222   | 0.048 | 0.064 | 0.848  | 0.130   | -0.029  |
| 18       | 0.001  | 0.189     | -0.679  | 0.416   | 0.143 | 0.189 | 0.845  | 0.127   | -0.050  |
| 48       | 0.020  | 0.197     | -0.426  | 0.794   | 0.145 | 0.198 | 0.831  | 0.119   | -0.166  |
| 90       | -0.049 | 0.141     | -0.437  | 0.240   | 0.110 | 0.149 | 0.909  | 0.253   | 0.087   |
| 150      | 0.030  | 0.129     | -0.264  | 0.365   | 0.106 | 0.133 | 0.897  | 0.204   | 0.171   |

```

In [14]: # Average value of each factor over the whole sample
mean_betas = factors.mean().values
# Nelson-Siegel curve built from the average factors, on the fine maturity grid up
mean_curve = ns_loadings(grid[show], LAMBDA) @ mean_betas

# Create a figure with one set of axes, 8 by 5 inches
fig, ax = plt.subplots(figsize=(8, 5))
# Draw the fitted average curve as a solid black line
ax.plot(grid[show], mean_curve, color="black", linewidth=1.5, label="Fitted Nelson-
# Mark the actual average yields with plus signs
ax.plot(tau, yields.mean().values, "+", color="black", markersize=10, markeredgewidth
# Label the horizontal axis
ax.set_xlabel("Maturity (months)")
# Label the vertical axis
ax.set_ylabel("Yield (percent)")
# Add a legend without a box
ax.legend(frameon=False)
# Add light grey grid lines
ax.grid(color="lightgrey", linewidth=0.5)
# Remove the top and right borders
ax.spines[["top", "right"]].set_visible(False)
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig4_average_fit.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()

```



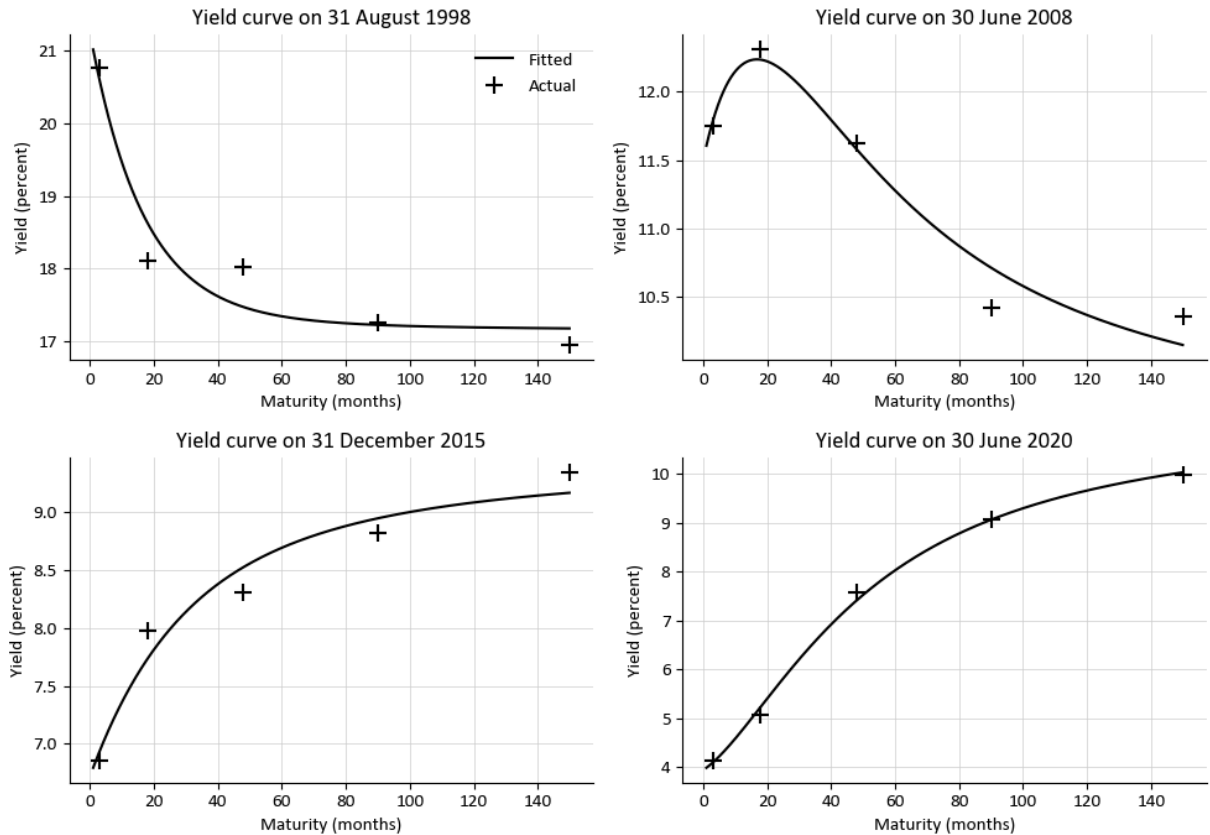
```
In [15]: # Four dates chosen to show different market conditions and curve shapes
DATES = ["1998-08-31", # emerging-market crisis
        "2008-06-30", # peak of the pre-crisis tightening cycle
        "2015-12-31", # finance minister change ("Nenegate")
        "2020-06-30"] # COVID-19 rate cuts

# Create a 2 by 2 grid of charts, 10 by 7 inches in total
fig, axes = plt.subplots(2, 2, figsize=(10, 7))
# Loop over each chart position together with its date
for ax, d in zip(axes.flat, DATES):
    # Convert the date text into a real date
    date = pd.Timestamp(d)
    # Fitted Nelson-Siegel curve for this month, on the fine maturity grid
    curve = ns_loadings(grid[show], LAMBDA) @ factors.loc[date].values
    # Draw the fitted curve as a solid black line
    ax.plot(grid[show], curve, color="black", linewidth=1.5, label="Fitted")
    # Mark the actual yields for this month with plus signs
    ax.plot(tau, yields.loc[date].values, "+", color="black", markersize=10, marker)
    # Give the chart a title showing the date
    ax.set_title("Yield curve on " + date.strftime("%d %B %Y"))
    # Label the horizontal axis
    ax.set_xlabel("Maturity (months)")
    # Label the vertical axis
    ax.set_ylabel("Yield (percent)")
    # Add light grey grid lines
    ax.grid(color="lightgrey", linewidth=0.5)
    # Remove the top and right borders
    ax.spines[["top", "right"]].set_visible(False)

# Add a Legend to the first chart only, to avoid repetition
axes.flat[0].legend(frameon=False)
# Adjust spacing so titles and labels do not overlap
```

```
fig.tight_layout()
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig5_selected_fits.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()

# Print the actual yields on the four selected dates, so the values behind the figure
print(yields.loc[pd.to_datetime(DATES)].round(2))
```



|            | y_3m  | y_18m | y_48m | y_90m | y_150m |
|------------|-------|-------|-------|-------|--------|
| 1998-08-31 | 20.77 | 18.11 | 18.02 | 17.25 | 16.95  |
| 2008-06-30 | 11.75 | 12.31 | 11.62 | 10.42 | 10.35  |
| 2015-12-31 | 6.85  | 7.98  | 8.31  | 8.82  | 9.34   |
| 2020-06-30 | 4.14  | 5.08  | 7.58  | 9.06  | 9.97   |

## Step 8: Factor properties and AR(1) dynamics

Estimated factors are compared with the empirical level, slope and curvature (Diebold and Li's Figure 7). Table 3 reports factor statistics and ADF (Augmented Dickey-Fuller) unit root tests, with lags chosen by the BIC (Bayesian information criterion), as in the paper. Each factor is then modelled as an AR(1) process,

$$\hat{\beta}_{it} = c_i + \gamma_i \hat{\beta}_{i,t-1} + \eta_{it}, \quad i = 1, 2, 3,$$

and the residual autocorrelations check whether the AR(1) captures the factor dynamics (their Figure 8).

```

In [16]: # Import the adfuller function from statsmodels, which runs the Augmented Dickey-Fu
from statsmodels.tsa.stattools import adfuller
# Import the main statsmodels module, used here for OLS regressions
import statsmodels.api as sm

# Scaling factor for beta3, so it plots on the same scale as the empirical curvature
# (Diebold and Li scaled beta3 by 0.3 for the same reason)
k3 = curv_emp.std() / factors["beta3"].std()
# Print the scaling factor so its value appears in the output
print("Scaling applied to beta3 in the figure:", round(k3, 3))

# Create three charts stacked vertically, sharing the same time axis
fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=True)

# Top chart: estimated level factor as a solid line
axes[0].plot(factors.index, factors["beta1"], color="black", linewidth=1.2, label="")
# Top chart: empirical level (150-month yield) as a dotted line
axes[0].plot(level_emp.index, level_emp, color="black", linewidth=1.2, linestyle=":")

# Middle chart: minus the estimated slope factor, so it moves in the same direction
axes[1].plot(factors.index, -factors["beta2"], color="black", linewidth=1.2, label="")
# Middle chart: empirical slope (150-month minus 3-month yield) as a dotted line
axes[1].plot(slope_emp.index, slope_emp, color="black", linewidth=1.2, linestyle=":")

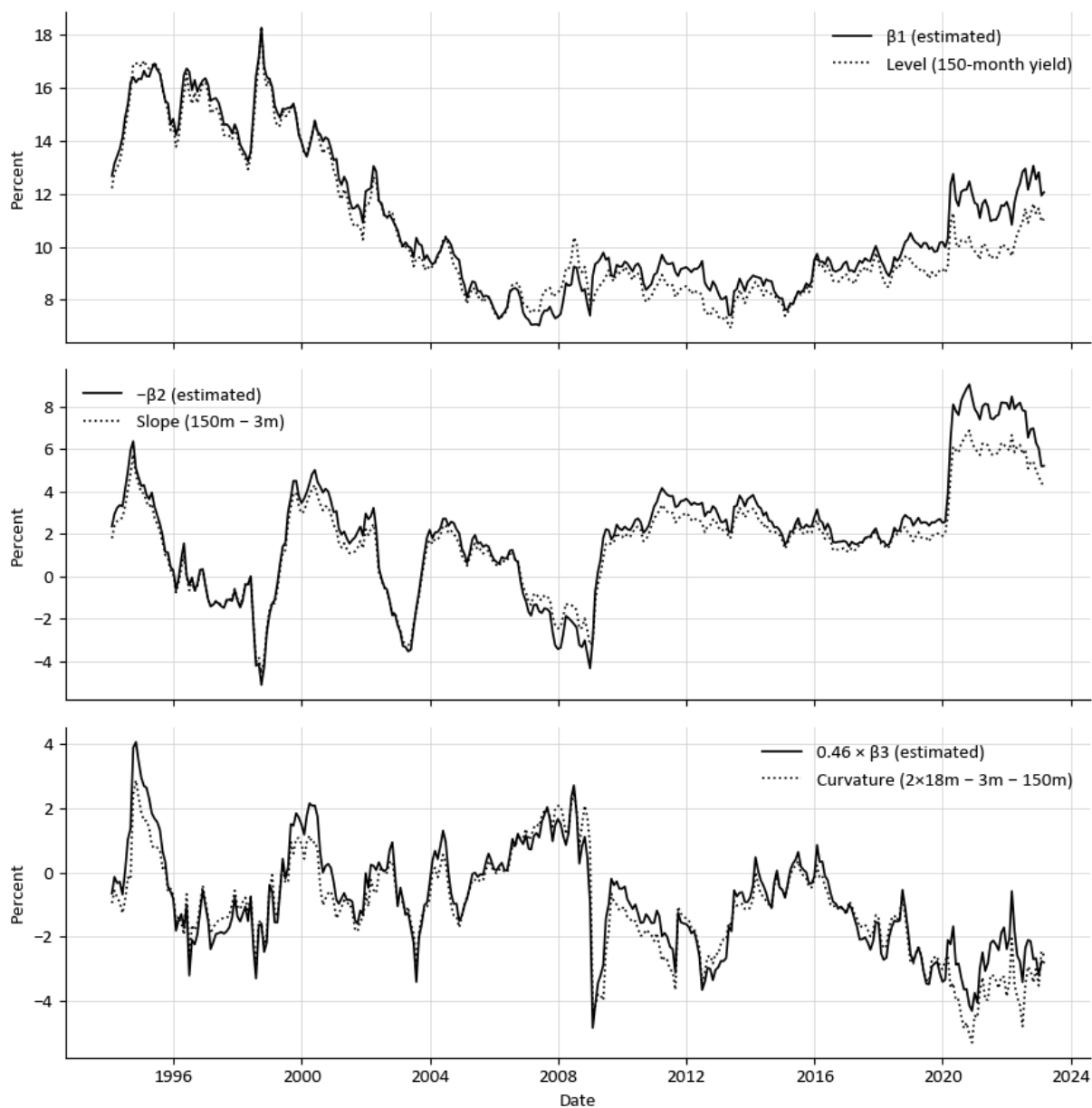
# Bottom chart: scaled estimated curvature factor as a solid line
axes[2].plot(factors.index, k3 * factors["beta3"], color="black", linewidth=1.2, label="")
# Bottom chart: empirical curvature as a dotted line
axes[2].plot(curv_emp.index, curv_emp, color="black", linewidth=1.2, linestyle=":")

# Apply the same formatting to all three charts
for ax in axes:
    # Label the vertical axis
    ax.set_ylabel("Percent")
    # Add a legend without a box, in the top right corner
    ax.legend(frameon=False, loc="best")
    # Add light grey grid lines
    ax.grid(color="lightgrey", linewidth=0.5)
    # Remove the top and right borders
    ax.spines[["top", "right"]].set_visible(False)

# Label the shared time axis on the bottom chart
axes[2].set_xlabel("Date")
# Adjust spacing so labels do not overlap
fig.tight_layout()
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig6_factors_vs_proxies.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()

```

Scaling applied to beta3 in the figure: 0.458



```
In [17]: # Empty list that will hold one row of statistics for each factor
rows = []
# Loop over each factor column
for col in factors.columns:
    # Pick out the estimated values of this factor
    s = factors[col]
    # Calculate the sample autocorrelations from lag 0 up to lag 30
    r = acf(s, nlags=30)
    # Run the ADF unit root test with a constant, choosing the number of lags by BIC
    adf = adfuller(s, regression="c", autolag="BIC")
    # Store the statistics for this factor as one row
    rows.append({
        "Factor": col,                # factor name
        "Mean": s.mean(),             # average value
        "Std. Dev.": s.std(),         # standard deviation
        "Minimum": s.min(),           # lowest value
        "Maximum": s.max(),           # highest value
        "rho(1)": r[1],               # autocorrelation at lag 1
    })
```

```

        "rho(12)": r[12],          # autocorrelation at lag 12
        "rho(30)": r[30],          # autocorrelation at lag 30
        "ADF": adf[0],             # ADF test statistic
        "ADF p-value": adf[1],     # p-value of the ADF test
        "ADF lags": adf[2],        # number of lags chosen by BIC
    })

# Turn the rows into a table, with the factor name as the row label
table3 = pd.DataFrame(rows).set_index("Factor")
# Print a title for the table
print("Table 3: Descriptive statistics, estimated factors, 1994:01 to 2023:02")
# Print the table rounded to 3 decimal places
print(table3.round(3))
# Print the ADF critical values at the 1%, 5% and 10% levels, as plain rounded numb
print("ADF critical values:", {k: float(round(v, 3)) for k, v in adf[4].items()})

```

Table 3: Descriptive statistics, estimated factors, 1994:01 to 2023:02

|        | Mean   | Std. Dev. | Minimum | Maximum | rho(1) | rho(12) | rho(30) | ADF    | ADF p-value |
|--------|--------|-----------|---------|---------|--------|---------|---------|--------|-------------|
| Factor |        |           |         |         |        |         |         |        |             |
| beta1  | 10.925 | 2.777     | 7.027   | 18.260  | 0.989  | 0.857   | 0.665   | -1.696 |             |
| 0.433  |        | 1         |         |         |        |         |         |        |             |
| beta2  | -2.057 | 2.802     | -9.044  | 5.126   | 0.980  | 0.537   | 0.011   | -2.581 |             |
| 0.097  |        | 1         |         |         |        |         |         |        |             |
| beta3  | -1.923 | 3.410     | -10.544 | 8.872   | 0.927  | 0.328   | 0.063   | -3.465 |             |
| 0.009  |        | 0         |         |         |        |         |         |        |             |

ADF critical values: {'1%': -3.449, '5%': -2.87, '10%': -2.571}

```

In [18]: # Dictionary that will store the AR(1) residuals for each factor
ar1_resid = {}
# Loop over each factor column
for col in factors.columns:
    # Dependent variable: the factor from the second month onwards
    y = factors[col].iloc[1:]
    # Explanatory variable: the factor one month earlier, plus a constant term
    x = sm.add_constant(factors[col].shift(1).iloc[1:].rename(col + "_lag1"))
    # Estimate the AR(1) regression by OLS
    model = sm.OLS(y, x).fit()
    # Print the full regression output for this factor
    print(model.summary())
    # Store the residuals of this AR(1) model
    ar1_resid[col] = model.resid

# Lags (displacements) from 1 to 60 months, used on the horizontal axis
LAGS = np.arange(1, 61)
# Create a grid of charts: 3 rows (one per factor) by 2 columns (factor, then AR(1))
fig, axes = plt.subplots(3, 2, figsize=(11, 10))
# Loop over each factor, keeping count with i (0, 1, 2)
for i, col in enumerate(factors.columns):
    # Two charts per factor: the factor's autocorrelations, and its AR(1) residuals
    panels = [
        (axes[i, 0], acf(factors[col], nlags=60)[1:], len(factors), f"Autocorrelati"),
        (axes[i, 1], acf(ar1_resid[col], nlags=60)[1:], len(ar1_resid[col]), f"Auto")
    ]
    # Loop over the two charts for this factor
    for ax, r, n, title in panels:

```

```
# Draw the autocorrelations as thin black bars
ax.bar(LAGS, r, color="black", width=0.6)
# Approximate 95% confidence band (Bartlett), equal to 1.96 divided by the
band = 1.96 / np.sqrt(n)
# Draw the upper confidence band as a dashed line
ax.axhline(band, color="black", linewidth=0.8, linestyle="--")
# Draw the lower confidence band as a dashed line
ax.axhline(-band, color="black", linewidth=0.8, linestyle="--")
# Draw a solid line at zero
ax.axhline(0, color="black", linewidth=0.8)
# Give the chart a title
ax.set_title(title)
# Label the horizontal axis
ax.set_xlabel("Displacement (months)")
# Label the vertical axis
ax.set_ylabel("Autocorrelation")
# Remove the top and right borders
ax.spines[["top", "right"]].set_visible(False)

# Adjust spacing so titles and labels do not overlap
fig.tight_layout()
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig7_factor_autocorrelations.png", dpi=300, bbox_inches="tight")
# Show the figure in the notebook
plt.show()
```

## OLS Regression Results

```

=====
Dep. Variable:          beta1    R-squared:          0.980
Model:                  OLS      Adj. R-squared:       0.980
Method:                 Least Squares    F-statistic:       1.728e+04
Date:                  Sun, 27 Sep 2026    Prob (F-statistic): 4.71e-298
Time:                  19:47:30    Log-Likelihood:    -166.02
No. Observations:      349    AIC:                336.0
Df Residuals:          347    BIC:                343.7
Df Model:              1
Covariance Type:       nonrobust
=====

```

```

=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
const          0.1099     0.085      1.295     0.196    -0.057     0.277
beta1_lag1     0.9898     0.008    131.451     0.000     0.975     1.005
=====
Omnibus:              71.551    Durbin-Watson:          1.581
Prob(Omnibus):         0.000    Jarque-Bera (JB):        245.125
Skew:                  0.874    Prob(JB):                5.91e-54
Kurtosis:              6.715    Cond. No.                 46.1
=====

```

## Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

## OLS Regression Results

```

=====
Dep. Variable:          beta2    R-squared:          0.965
Model:                  OLS      Adj. R-squared:       0.964
Method:                 Least Squares    F-statistic:       9451.
Date:                  Sun, 27 Sep 2026    Prob (F-statistic): 8.36e-254
Time:                  19:47:30    Log-Likelihood:    -271.81
No. Observations:      349    AIC:                547.6
Df Residuals:          347    BIC:                555.3
Df Model:              1
Covariance Type:       nonrobust
=====

```

```

=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
const         -0.0411     0.035     -1.173     0.242    -0.110     0.028
beta2_lag1     0.9839     0.010     97.218     0.000     0.964     1.004
=====
Omnibus:              40.690    Durbin-Watson:          1.208
Prob(Omnibus):         0.000    Jarque-Bera (JB):        221.984
Skew:                 -0.231    Prob(JB):                6.26e-49
Kurtosis:              6.880    Cond. No.                 4.43
=====

```

## Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

## OLS Regression Results

```

=====
Dep. Variable:          beta3    R-squared:          0.863
Model:                  OLS      Adj. R-squared:       0.863

```

```

Method:                Least Squares      F-statistic:                2188.
Date:                  Sun, 27 Sep 2026    Prob (F-statistic):         6.59e-152
Time:                  19:47:30           Log-Likelihood:             -576.26
No. Observations:      349               AIC:                        1157.
Df Residuals:          347               BIC:                        1164.
Df Model:              1
Covariance Type:       nonrobust

```

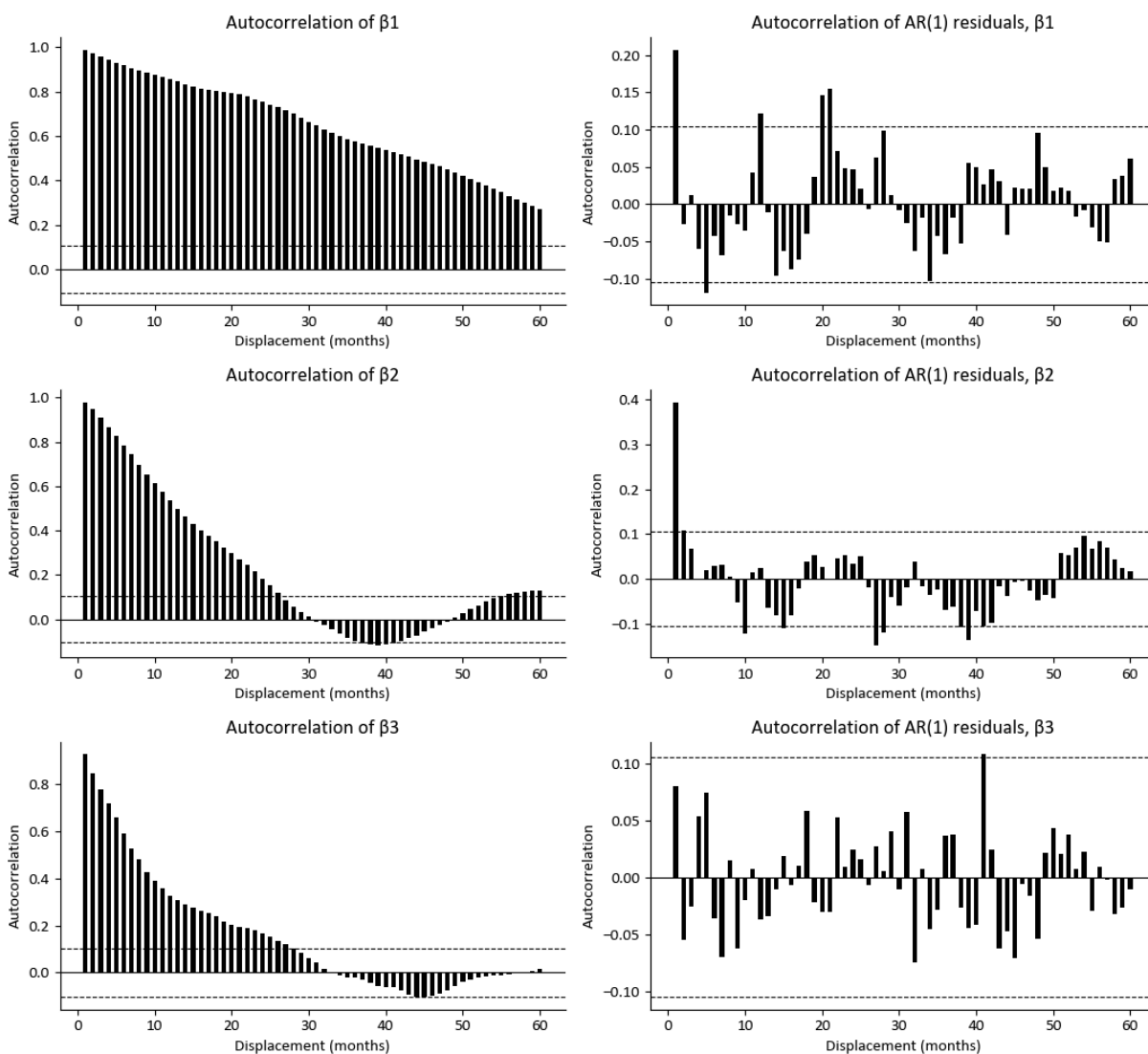
```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----+-----
const        -0.1453      0.078      -1.870      0.062     -0.298      0.007
beta3_lag1     0.9310      0.020     46.776      0.000      0.892      0.970
=====
Omnibus:                74.887    Durbin-Watson:                1.838
Prob(Omnibus):           0.000    Jarque-Bera (JB):             759.453
Skew:                   -0.532    Prob(JB):                     1.22e-165
Kurtosis:               10.148    Cond. No.                     4.55
=====

```

## Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



## Step 9: Out-of-sample forecasting

Forecasts are made recursively from January 2003, using only data available at each forecast date, for horizons  $h = 1, 6$  and  $12$  months. Following Diebold and Li (footnote 11), all models forecast directly by regressing the value at  $t + h$  on the value at  $t$ .

**Nelson-Siegel with AR(1) factors:**  $\hat{\beta}_{i,t+h/t} = \hat{c}_i + \hat{\gamma}_i \hat{\beta}_{it}$ , then

$$\hat{y}_{t+h/t}(\tau) = \hat{\beta}_{1,t+h/t} + \hat{\beta}_{2,t+h/t} \left( \frac{1-e^{-\lambda\tau}}{\lambda\tau} \right) + \hat{\beta}_{3,t+h/t} \left( \frac{1-e^{-\lambda\tau}}{\lambda\tau} - e^{-\lambda\tau} \right)$$

**Competitors:** Nelson-Siegel with VAR(1) factors; random walk ( $\hat{y}_{t+h/t} = y_t$ ); AR(1) on each yield; VAR(1) on all yields; slope regression ( $\hat{y}_{t+h/t}(\tau) - y_t(\tau) = \hat{c} + \hat{\gamma}(y_t(\tau) - y_t(3))$ ). The Fama-Bliss and Cochrane-Piazzesi forward-rate regressions are not estimated, because banded yields do not provide forward rates at the required maturities.

```
In [19]: # Define a function that runs a direct OLS forecasting regression and returns the f
# Y: the values h months ahead (target), Z: the values today (explanatory), z_now:
def direct_forecast(Y, Z, z_now):
    # Add a column of ones to Z, so the regression includes a constant term
    Zc = np.column_stack([np.ones(len(Z)), Z])
    # Estimate the regression coefficients by OLS
    B = np.linalg.lstsq(Zc, Y, rcond=None)[0]
    # Forecast: constant plus coefficients times today's value
    return np.concatenate([[1.0], np.atleast_1d(z_now)]) @ B

# Forecast horizons in months, as in Diebold and Li
HORIZONS = [1, 6, 12]
# Date of the first forecast: after a 9-year estimation window (1994 to 2002)
FIRST_ORIGIN = pd.Timestamp("2003-01-31")
# Names of the models being compared
MODELS = ["NS-AR(1)", "NS-VAR(1)", "Random walk", "AR(1) on yields", "VAR(1) on yie

# Yields as a plain array of numbers (rows = months, columns = maturities)
yv = yields.values
# Estimated factors as a plain array of numbers (rows = months, columns = beta1, be
fv = factors.values
# Number of maturities in the main specification (5)
n_mat = yv.shape[1]
# Row position of the first forecast date
i0 = yields.index.get_loc(FIRST_ORIGIN)

# Dictionary that will hold all forecasts, organised by horizon and then by model
forecasts = {}
# Loop over each forecast horizon
for h in HORIZONS:
    # Empty list of forecasts for each model at this horizon
    store = {m: [] for m in MODELS}
    # Empty list of the dates being forecast (the target dates)
    targets = []
    # Loop over each forecast date i, stopping h months before the end so the actua
    for i in range(i0, len(yields) - h):
```

```

# Factor pairs available at date i: today's factors (Zf) and the factors h
Zf, Yf = fv[:, i - h + 1], fv[h : i + 1]
# Yield pairs available at date i: today's yields (Zy) and the yields h mon
Zy, Yy = yv[:, i - h + 1], yv[h : i + 1]

# NS-AR(1): forecast each factor separately from its own current value
b_ar = np.array([direct_forecast(Yf[:, j], Zf[:, [j]], fv[i, j]) for j in range(1, n_mat)])
# Turn the forecast factors into forecast yields using the Nelson-Siegel Lo
store["NS-AR(1)"].append(X @ b_ar)

# NS-VAR(1): forecast all three factors jointly from all three current fact
b_var = direct_forecast(Yf, Zf, fv[i])
# Turn the forecast factors into forecast yields
store["NS-VAR(1)"].append(X @ b_var)

# Random walk: the forecast is simply today's yield ("no change")
store["Random walk"].append(yv[i].copy())

# AR(1) on yields: forecast each yield separately from its own current value
store["AR(1) on yields"].append(np.array([direct_forecast(Yy[:, j], Zy[:, [j]], yv[i, j]) for j in range(1, n_mat)]))

# VAR(1) on yields: forecast all five yields jointly from all five current
store["VAR(1) on yields"].append(direct_forecast(Yy, Zy, yv[i]))

# Slope regression: not defined for the 3-month yield, so start with a miss
slope_fc = [np.nan]
# Loop over the longer maturities (18 to 150 months)
for j in range(1, n_mat):
    # Historical yield changes over h months for this maturity
    dy = Yy[:, j] - Zy[:, j]
    # Historical slope: this maturity's yield minus the 3-month yield
    sp = Zy[:, j] - Zy[:, 0]
    # Forecast: today's yield plus the predicted change, based on today's slope
    slope_fc.append(yv[i, j] + direct_forecast(dy, sp[:, None], yv[i, j] - Zy[i, 0]))
# Store the slope regression forecasts
store["Slope regression"].append(np.array(slope_fc))

# Record the date being forecast (h months after date i)
targets.append(yields.index[i + h])

# Turn each model's list of forecasts into a table with target dates as rows and
forecasts[h] = {m: pd.DataFrame(np.vstack(v), index=targets, columns=MAIN_COLS)}

# Print the number of forecasts and the range of target dates for each horizon
for h in HORIZONS:
    # First and last target date at this horizon
    first, last = forecasts[h]["Random walk"].index[[0, -1]]
    # Print a one-line summary
    print(f"h = {h:2d}: {len(forecasts[h]['Random walk'])} forecasts, targets {first} to {last}")

```

h = 1: 241 forecasts, targets 2003-02-28 to 2023-02-28  
h = 6: 236 forecasts, targets 2003-07-31 to 2023-02-28  
h = 12: 230 forecasts, targets 2004-01-31 to 2023-02-28

In [20]: # Autocorrelation lags reported for the forecast errors at each horizon, as in Diebold et al. (2003)  
ACF\_LAGS = {1: (1, 12), 6: (6, 18), 12: (12, 24)}

```

# Dictionary that will hold the forecast errors, by horizon and then by model
errors = {}
# Dictionary that will hold a compact RMSE table for each horizon
rmse_tables = {}

# Loop over each forecast horizon
for h in HORIZONS:
    # The two autocorrelation lags to report at this horizon
    la, lb = ACF_LAGS[h]
    # Empty dictionary of errors for this horizon
    errors[h] = {}
    # Empty list that will hold one row per model and maturity
    rows = []
    # Loop over each model
    for m in MODELS:
        # This model's forecasts at this horizon
        fc = forecasts[h][m]
        # Forecast errors: actual yield minus forecast yield, on the target dates
        err = yields.loc[fc.index] - fc
        # Store the errors for the Diebold-Mariano tests later
        errors[h][m] = err
        # Loop over each maturity
        for col in MAIN_COLS:
            # Pick out the errors for this maturity
            e = err[col]
            # Skip maturities where the model gives no forecast (slope regression a
            if e.isna().all():
                continue
            # Autocorrelations of the forecast errors up to the longer reported lag
            r = acf(e, nlags=lb)
            # Store the statistics for this model and maturity
            rows.append({
                "Model": m,                                # model name
                "Maturity": col,                            # maturity
                "Mean": e.mean(),                          # average error (bias)
                "Std. Dev.": e.std(),                      # standard deviation of er
                "RMSE": np.sqrt((e ** 2).mean()),          # root mean squared error
                f"rho({la})": r[la],                       # error autocorrelation at
                f"rho({lb})": r[lb],                       # error autocorrelation at
            })

    # Turn the rows into a table, labelled by model and maturity
    table = pd.DataFrame(rows).set_index(["Model", "Maturity"])
    # Print a title matching Diebold and Li's table numbering (Table 4 for h=1, 5 f
    print(f"\nTable {4 + HORIZONS.index(h)}: Out-of-sample {h}-month-ahead forecast
    # Print the full table, rounded to 3 decimal places
    print(table.round(3).to_string())
    # Keep a compact version: RMSE only, with models as rows and maturities as colu
    rmse_tables[h] = table["RMSE"].unstack("Maturity")[MAIN_COLS].reindex(MODELS)

# Print the compact RMSE comparison tables
for h in HORIZONS:
    # Print a title
    print(f"\nRMSE, {h}-month-ahead forecasts (percentage points)")
    # Print the RMSE of every model at every maturity
    print(rmse_tables[h].round(3).to_string())

```

```
# Print a title for the relative table
print(f"\nRMSE relative to random walk, {h}-month-ahead (below 1 = more accurat
# Divide every model's RMSE by the random walk's RMSE at the same maturity
print((rmse_tables[h] / rmse_tables[h].loc["Random walk"]).round(3).to_string()
```

Table 4: Out-of-sample 1-month-ahead forecasting results

| Model            | Maturity | Mean   | Std. Dev. | RMSE  | rho(1) | rho(12) |
|------------------|----------|--------|-----------|-------|--------|---------|
| NS-AR(1)         | y_3m     | -0.063 | 0.290     | 0.297 | 0.417  | 0.144   |
|                  | y_18m    | -0.038 | 0.434     | 0.435 | 0.568  | 0.107   |
|                  | y_48m    | -0.041 | 0.378     | 0.380 | 0.372  | 0.054   |
|                  | y_90m    | -0.101 | 0.357     | 0.371 | 0.345  | 0.183   |
|                  | y_150m   | 0.046  | 0.333     | 0.335 | 0.301  | 0.119   |
| NS-VAR(1)        | y_3m     | 0.015  | 0.268     | 0.267 | 0.338  | 0.099   |
|                  | y_18m    | 0.008  | 0.429     | 0.428 | 0.536  | 0.122   |
|                  | y_48m    | -0.018 | 0.389     | 0.389 | 0.383  | 0.090   |
|                  | y_90m    | -0.086 | 0.372     | 0.381 | 0.377  | 0.230   |
|                  | y_150m   | 0.058  | 0.340     | 0.344 | 0.315  | 0.151   |
| Random walk      | y_3m     | -0.025 | 0.294     | 0.294 | 0.405  | 0.096   |
|                  | y_18m    | -0.014 | 0.366     | 0.365 | 0.279  | -0.009  |
|                  | y_48m    | -0.012 | 0.346     | 0.346 | 0.243  | 0.069   |
|                  | y_90m    | -0.003 | 0.329     | 0.328 | 0.174  | 0.077   |
|                  | y_150m   | 0.004  | 0.302     | 0.301 | 0.167  | 0.157   |
| AR(1) on yields  | y_3m     | -0.047 | 0.299     | 0.302 | 0.426  | 0.107   |
|                  | y_18m    | -0.043 | 0.370     | 0.372 | 0.296  | 0.004   |
|                  | y_48m    | -0.035 | 0.351     | 0.352 | 0.262  | 0.080   |
|                  | y_90m    | -0.010 | 0.331     | 0.331 | 0.183  | 0.079   |
|                  | y_150m   | -0.000 | 0.303     | 0.303 | 0.173  | 0.158   |
| VAR(1) on yields | y_3m     | 0.018  | 0.287     | 0.287 | 0.394  | 0.120   |
|                  | y_18m    | 0.013  | 0.400     | 0.399 | 0.410  | 0.097   |
|                  | y_48m    | -0.024 | 0.374     | 0.374 | 0.325  | 0.124   |
|                  | y_90m    | -0.017 | 0.355     | 0.355 | 0.267  | 0.151   |
|                  | y_150m   | -0.003 | 0.325     | 0.325 | 0.247  | 0.213   |
| Slope regression | y_18m    | -0.002 | 0.365     | 0.365 | 0.254  | -0.015  |
|                  | y_48m    | 0.000  | 0.348     | 0.347 | 0.231  | 0.073   |
|                  | y_90m    | 0.010  | 0.331     | 0.330 | 0.167  | 0.079   |
|                  | y_150m   | 0.015  | 0.304     | 0.303 | 0.164  | 0.163   |

Table 5: Out-of-sample 6-month-ahead forecasting results

| Model           | Maturity | Mean   | Std. Dev. | RMSE  | rho(6) | rho(18) |
|-----------------|----------|--------|-----------|-------|--------|---------|
| NS-AR(1)        | y_3m     | -0.607 | 1.286     | 1.420 | 0.516  | 0.096   |
|                 | y_18m    | -0.540 | 1.321     | 1.424 | 0.392  | 0.141   |
|                 | y_48m    | -0.412 | 1.037     | 1.114 | 0.113  | 0.090   |
|                 | y_90m    | -0.326 | 0.895     | 0.951 | -0.002 | 0.054   |
|                 | y_150m   | -0.088 | 0.825     | 0.828 | -0.084 | 0.026   |
| NS-VAR(1)       | y_3m     | -0.296 | 1.267     | 1.298 | 0.409  | 0.054   |
|                 | y_18m    | -0.315 | 1.389     | 1.422 | 0.401  | 0.088   |
|                 | y_48m    | -0.257 | 1.149     | 1.176 | 0.262  | 0.058   |
|                 | y_90m    | -0.202 | 1.009     | 1.027 | 0.194  | 0.034   |
|                 | y_150m   | 0.022  | 0.910     | 0.908 | 0.082  | -0.014  |
| Random walk     | y_3m     | -0.162 | 1.142     | 1.151 | 0.226  | -0.006  |
|                 | y_18m    | -0.085 | 1.107     | 1.108 | -0.117 | 0.062   |
|                 | y_48m    | -0.050 | 0.939     | 0.939 | -0.274 | 0.036   |
|                 | y_90m    | 0.006  | 0.814     | 0.812 | -0.315 | 0.014   |
|                 | y_150m   | 0.036  | 0.740     | 0.739 | -0.337 | -0.019  |
| AR(1) on yields | y_3m     | -0.647 | 1.195     | 1.356 | 0.400  | 0.077   |
|                 | y_18m    | -0.444 | 1.161     | 1.241 | 0.050  | 0.102   |
|                 | y_48m    | -0.366 | 1.034     | 1.095 | -0.014 | 0.121   |
|                 | y_90m    | -0.159 | 0.884     | 0.896 | -0.132 | 0.091   |
|                 | y_150m   | -0.098 | 0.795     | 0.799 | -0.161 | 0.084   |

|                  |        |        |       |       |        |        |
|------------------|--------|--------|-------|-------|--------|--------|
| VAR(1) on yields | y_3m   | -0.628 | 1.560 | 1.679 | 0.345  | 0.103  |
|                  | y_18m  | -0.591 | 1.522 | 1.630 | 0.247  | 0.055  |
|                  | y_48m  | -0.552 | 1.323 | 1.431 | 0.301  | 0.089  |
|                  | y_90m  | -0.384 | 1.138 | 1.199 | 0.243  | 0.057  |
|                  | y_150m | -0.296 | 1.056 | 1.095 | 0.208  | 0.070  |
| Slope regression | y_18m  | 0.016  | 1.121 | 1.118 | -0.171 | 0.067  |
|                  | y_48m  | 0.084  | 0.962 | 0.964 | -0.274 | 0.033  |
|                  | y_90m  | 0.145  | 0.826 | 0.837 | -0.323 | 0.003  |
|                  | y_150m | 0.189  | 0.746 | 0.768 | -0.344 | -0.036 |

Table 6: Out-of-sample 12-month-ahead forecasting results

|                  |          | Mean   | Std. Dev. | RMSE  | rho(12) | rho(24) |
|------------------|----------|--------|-----------|-------|---------|---------|
| Model            | Maturity |        |           |       |         |         |
| NS-AR(1)         | y_3m     | -0.989 | 2.150     | 2.362 | 0.516   | -0.009  |
|                  | y_18m    | -0.781 | 1.998     | 2.142 | 0.494   | 0.025   |
|                  | y_48m    | -0.483 | 1.471     | 1.545 | 0.363   | 0.036   |
|                  | y_90m    | -0.282 | 1.157     | 1.188 | 0.126   | 0.019   |
|                  | y_150m   | 0.026  | 1.025     | 1.023 | 0.053   | 0.041   |
| NS-VAR(1)        | y_3m     | -0.546 | 2.236     | 2.297 | 0.389   | -0.182  |
|                  | y_18m    | -0.460 | 2.084     | 2.130 | 0.358   | -0.158  |
|                  | y_48m    | -0.246 | 1.567     | 1.583 | 0.289   | -0.108  |
|                  | y_90m    | -0.073 | 1.296     | 1.295 | 0.161   | -0.067  |
|                  | y_150m   | 0.223  | 1.138     | 1.157 | 0.113   | -0.023  |
| Random walk      | y_3m     | -0.277 | 1.691     | 1.710 | 0.110   | -0.218  |
|                  | y_18m    | -0.154 | 1.430     | 1.435 | -0.045  | -0.133  |
|                  | y_48m    | -0.060 | 1.101     | 1.100 | -0.162  | 0.016   |
|                  | y_90m    | 0.030  | 0.945     | 0.943 | -0.372  | 0.206   |
|                  | y_150m   | 0.073  | 0.851     | 0.852 | -0.323  | 0.210   |
| AR(1) on yields  | y_3m     | -1.256 | 1.767     | 2.165 | 0.294   | -0.168  |
|                  | y_18m    | -0.640 | 1.520     | 1.646 | 0.108   | -0.164  |
|                  | y_48m    | -0.413 | 1.276     | 1.338 | 0.103   | -0.002  |
|                  | y_90m    | -0.099 | 1.059     | 1.061 | -0.091  | 0.159   |
|                  | y_150m   | 0.004  | 0.934     | 0.932 | -0.050  | 0.175   |
| VAR(1) on yields | y_3m     | -0.966 | 2.288     | 2.479 | 0.130   | -0.139  |
|                  | y_18m    | -0.782 | 2.057     | 2.197 | 0.078   | -0.176  |
|                  | y_48m    | -0.510 | 1.669     | 1.742 | 0.165   | -0.106  |
|                  | y_90m    | -0.307 | 1.389     | 1.419 | 0.105   | -0.034  |
|                  | y_150m   | -0.102 | 1.231     | 1.232 | 0.134   | -0.005  |
| Slope regression | y_18m    | 0.173  | 1.467     | 1.474 | -0.036  | -0.132  |
|                  | y_48m    | 0.343  | 1.137     | 1.186 | -0.089  | -0.024  |
|                  | y_90m    | 0.451  | 0.945     | 1.045 | -0.322  | 0.163   |
|                  | y_150m   | 0.544  | 0.818     | 0.982 | -0.297  | 0.195   |

RMSE, 1-month-ahead forecasts (percentage points)

| Maturity         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|------------------|-------|-------|-------|-------|--------|
| Model            |       |       |       |       |        |
| NS-AR(1)         | 0.297 | 0.435 | 0.380 | 0.371 | 0.335  |
| NS-VAR(1)        | 0.267 | 0.428 | 0.389 | 0.381 | 0.344  |
| Random walk      | 0.294 | 0.365 | 0.346 | 0.328 | 0.301  |
| AR(1) on yields  | 0.302 | 0.372 | 0.352 | 0.331 | 0.303  |
| VAR(1) on yields | 0.287 | 0.399 | 0.374 | 0.355 | 0.325  |
| Slope regression | NaN   | 0.365 | 0.347 | 0.330 | 0.303  |

RMSE relative to random walk, 1-month-ahead (below 1 = more accurate than random walk)

| Maturity | y_3m | y_18m | y_48m | y_90m | y_150m |
|----------|------|-------|-------|-------|--------|
|----------|------|-------|-------|-------|--------|

|                  |       |       |       |       |       |
|------------------|-------|-------|-------|-------|-------|
| Model            |       |       |       |       |       |
| NS-AR(1)         | 1.007 | 1.190 | 1.098 | 1.129 | 1.113 |
| NS-VAR(1)        | 0.908 | 1.173 | 1.124 | 1.161 | 1.143 |
| Random walk      | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 |
| AR(1) on yields  | 1.026 | 1.018 | 1.018 | 1.008 | 1.005 |
| VAR(1) on yields | 0.975 | 1.094 | 1.080 | 1.081 | 1.079 |
| Slope regression | NaN   | 0.999 | 1.004 | 1.006 | 1.008 |

RMSE, 6-month-ahead forecasts (percentage points)

|                  |       |       |       |       |        |
|------------------|-------|-------|-------|-------|--------|
| Maturity         | y_3m  | y_18m | y_48m | y_90m | y_150m |
| Model            |       |       |       |       |        |
| NS-AR(1)         | 1.420 | 1.424 | 1.114 | 0.951 | 0.828  |
| NS-VAR(1)        | 1.298 | 1.422 | 1.176 | 1.027 | 0.908  |
| Random walk      | 1.151 | 1.108 | 0.939 | 0.812 | 0.739  |
| AR(1) on yields  | 1.356 | 1.241 | 1.095 | 0.896 | 0.799  |
| VAR(1) on yields | 1.679 | 1.630 | 1.431 | 1.199 | 1.095  |
| Slope regression | NaN   | 1.118 | 0.964 | 0.837 | 0.768  |

RMSE relative to random walk, 6-month-ahead (below 1 = more accurate than random walk)

|                  |       |       |       |       |        |
|------------------|-------|-------|-------|-------|--------|
| Maturity         | y_3m  | y_18m | y_48m | y_90m | y_150m |
| Model            |       |       |       |       |        |
| NS-AR(1)         | 1.234 | 1.285 | 1.187 | 1.171 | 1.119  |
| NS-VAR(1)        | 1.128 | 1.283 | 1.252 | 1.265 | 1.229  |
| Random walk      | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |
| AR(1) on yields  | 1.179 | 1.120 | 1.166 | 1.104 | 1.081  |
| VAR(1) on yields | 1.459 | 1.471 | 1.525 | 1.477 | 1.481  |
| Slope regression | NaN   | 1.009 | 1.027 | 1.031 | 1.039  |

RMSE, 12-month-ahead forecasts (percentage points)

|                  |       |       |       |       |        |
|------------------|-------|-------|-------|-------|--------|
| Maturity         | y_3m  | y_18m | y_48m | y_90m | y_150m |
| Model            |       |       |       |       |        |
| NS-AR(1)         | 2.362 | 2.142 | 1.545 | 1.188 | 1.023  |
| NS-VAR(1)        | 2.297 | 2.130 | 1.583 | 1.295 | 1.157  |
| Random walk      | 1.710 | 1.435 | 1.100 | 0.943 | 0.852  |
| AR(1) on yields  | 2.165 | 1.646 | 1.338 | 1.061 | 0.932  |
| VAR(1) on yields | 2.479 | 2.197 | 1.742 | 1.419 | 1.232  |
| Slope regression | NaN   | 1.474 | 1.186 | 1.045 | 0.982  |

RMSE relative to random walk, 12-month-ahead (below 1 = more accurate than random walk)

|                  |       |       |       |       |        |
|------------------|-------|-------|-------|-------|--------|
| Maturity         | y_3m  | y_18m | y_48m | y_90m | y_150m |
| Model            |       |       |       |       |        |
| NS-AR(1)         | 1.382 | 1.492 | 1.404 | 1.260 | 1.200  |
| NS-VAR(1)        | 1.344 | 1.484 | 1.439 | 1.373 | 1.358  |
| Random walk      | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |
| AR(1) on yields  | 1.266 | 1.147 | 1.217 | 1.125 | 1.094  |
| VAR(1) on yields | 1.450 | 1.531 | 1.583 | 1.505 | 1.446  |
| Slope regression | NaN   | 1.027 | 1.078 | 1.108 | 1.152  |

```
In [21]: # Import the stats module from scipy, used for normal distribution p-values
         from scipy import stats

         # Define a function for the Diebold-Mariano test of equal forecast accuracy
         def diebold_mariano(e1, e2, h):
             # Loss differential: squared error of model 1 minus squared error of model 2, d
```

```

d = (e1 ** 2 - e2 ** 2).dropna()
# Regress the loss differential on a constant, with Newey-West (HAC) standard e
# because h-step-ahead forecast errors overlap and are autocorrelated
res = sm.OLS(d.values, np.ones(len(d))).fit(cov_type="HAC", cov_kwds={"maxlags"
# The t-statistic on the constant is the Diebold-Mariano statistic
dm = res.tvalues[0]
# Two-sided p-value from the standard normal distribution
p = 2 * (1 - stats.norm.cdf(abs(dm)))
# Return the statistic and its p-value
return dm, p

# Benchmark models that NS-AR(1) is tested against
BENCHMARKS = ["Random walk", "AR(1) on yields", "Slope regression"]
# Empty list that will hold one row per maturity
rows = []
# Loop over each maturity
for col in MAIN_COLS:
    # Start a new row for this maturity
    row = {"Maturity": col}
    # Loop over each horizon
    for h in HORIZONS:
        # Loop over each benchmark model
        for bm in BENCHMARKS:
            # NS-AR(1) forecast errors at this horizon and maturity
            e_ns = errors[h]["NS-AR(1)"][col]
            # Benchmark forecast errors at this horizon and maturity
            e_bm = errors[h][bm][col]
            # If the benchmark has no forecasts here (slope regression at 3 months)
            if e_bm.isna().all():
                row[f"h={h} vs {bm}"] = "NA"
                continue
            # Run the Diebold-Mariano test
            dm, p = diebold_mariano(e_ns, e_bm, h)
            # Mark significance: ** for the 5% level, * for the 10% level
            stars = "***" if p < 0.05 else ("*" if p < 0.10 else "")
            # Store the statistic with its significance marker
            row[f"h={h} vs {bm}"] = f"{dm:.2f}{stars}"
        # Add this maturity's row to the list
        rows.append(row)

# Turn the rows into a table, labelled by maturity
table7 = pd.DataFrame(rows).set_index("Maturity")
# Print a title for the table
print("Table 7: Diebold-Mariano tests, NS-AR(1) against benchmarks")
# Print an explanation of the sign convention
print("Negative values: NS-AR(1) more accurate. ** significant at 5%, * significant")
# Print the table with tests as rows and maturities as columns
print(table7.T.to_string())

```

Table 7: Diebold-Mariano tests, NS-AR(1) against benchmarks

Negative values: NS-AR(1) more accurate. \*\* significant at 5%, \* significant at 10%.

| Maturity                 | y_3m   | y_18m  | y_48m  | y_90m  | y_150m |
|--------------------------|--------|--------|--------|--------|--------|
| h=1 vs Random walk       | 0.37   | 3.37** | 2.60** | 3.21** | 3.30** |
| h=1 vs AR(1) on yields   | -0.82  | 3.14** | 2.02** | 3.07** | 3.14** |
| h=1 vs Slope regression  | NA     | 3.35** | 2.34** | 2.89** | 3.31** |
| h=6 vs Random walk       | 2.89** | 2.55** | 2.21** | 2.45** | 1.87*  |
| h=6 vs AR(1) on yields   | 0.60   | 1.57   | 0.20   | 1.12   | 0.73   |
| h=6 vs Slope regression  | NA     | 2.46** | 1.84*  | 1.78*  | 1.18   |
| h=12 vs Random walk      | 2.06** | 2.26** | 2.18** | 1.95*  | 1.63   |
| h=12 vs AR(1) on yields  | 0.54   | 1.56   | 0.87   | 0.94   | 0.86   |
| h=12 vs Slope regression | NA     | 2.21** | 1.71*  | 0.99   | 0.33   |

## Step 10: Forecast diagnostics

The random walk outperforms NS-AR(1) at all horizons, in contrast to Diebold and Li. Two explanations are tested:

1. **Fitting error:** the NS forecast starts from the fitted, not actual, curve. An *NS random walk* ( $\hat{\beta}_{t+h/t} = \hat{\beta}_t$ ) isolates this cost.
2. **Regime change:** recursive AR(1) estimates revert towards a mean that includes the high-rate pre-inflation-targeting period. A *rolling* 108-month (9-year) estimation window tests this.

Results are also reported for the sub-periods 2003 to 2012 and 2013 to 2023.

```
In [22]: # Length of the rolling estimation window in months (9 years, matching the initial
WINDOW = 108
# Names of the diagnostic models
NEW_MODELS = ["NS random walk", "NS-AR(1) rolling", "AR(1) on yields rolling"]

# Loop over each forecast horizon
for h in HORIZONS:
    # Empty list of forecasts for each diagnostic model
    store = {m: [] for m in NEW_MODELS}
    # Empty list of target dates
    targets = []
    # Loop over each forecast date i, as in Step 9
    for i in range(i0, len(yields) - h):
        # First row of the rolling window: keep only the most recent 108 forecastin
        start = max(0, i - h + 1 - WINDOW)
        # Rolling-window factor pairs: factors today (Zf) and h months Later (Yf)
        Zf, Yf = fv[start : i - h + 1], fv[start + h : i + 1]
        # Rolling-window yield pairs: yields today (Zy) and h months Later (Yy)
        Zy, Yy = yv[start : i - h + 1], yv[start + h : i + 1]

        # NS random walk: keep today's factors unchanged and rebuild the fitted cur
        store["NS random walk"].append(X @ fv[i])
        # NS-AR(1) rolling: forecast each factor using only the rolling window
        b_roll = np.array([direct_forecast(Yf[:, j], Zf[:, [j]], fv[i, j]) for j in
        # Turn the forecast factors into forecast yields
        store["NS-AR(1) rolling"].append(X @ b_roll)
```

```

# AR(1) on yields rolling: forecast each yield using only the rolling window
store["AR(1) on yields rolling"].append(np.array([direct_forecast(Yy[:, j],

# Record the date being forecast
targets.append(yields.index[i + h])

# Loop over each diagnostic model to store its forecasts and errors
for m, v in store.items():
    # Store the forecasts as a table, alongside the Step 9 models
    forecasts[h][m] = pd.DataFrame(np.vstack(v), index=targets, columns=MAIN_CO
    # Store the forecast errors (actual minus forecast), alongside the Step 9 m
    errors[h][m] = yields.loc[targets] - forecasts[h][m]

# Models to compare in the diagnostic tables
COMPARE = ["NS-AR(1)", "NS random walk", "NS-AR(1) rolling", "AR(1) on yields", "AR
# Evaluation periods: the full forecast period and two halves (dates refer to the d
PERIODS = {"Full 2003-2023": ("2003-01-01", "2023-12-31"),
           "2003-2012": ("2003-01-01", "2012-12-31"),
           "2013-2023": ("2013-01-01", "2023-12-31")}

# Loop over each horizon
for h in HORIZONS:
    # Loop over each evaluation period
    for label, (a, b) in PERIODS.items():
        # RMSE of each model at each maturity, using only forecasts whose target da
        rmse = pd.DataFrame({m: np.sqrt((errors[h][m].loc[a:b] ** 2).mean()) for m
        # Print a title
        print(f"\nRMSE relative to random walk, h = {h}, {label} (below 1 = more ac
        # Print each model's RMSE divided by the random walk's RMSE
        print((rmse / rmse.loc["Random walk"]).round(3).to_string())

```

RMSE relative to random walk, h = 1, Full 2003-2023 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.007 | 1.190 | 1.098 | 1.129 | 1.113  |
| NS random walk          | 0.952 | 1.118 | 1.106 | 1.095 | 1.102  |
| NS-AR(1) rolling        | 1.024 | 1.180 | 1.085 | 1.100 | 1.128  |
| AR(1) on yields         | 1.026 | 1.018 | 1.018 | 1.008 | 1.005  |
| AR(1) on yields rolling | 1.033 | 1.012 | 1.014 | 1.011 | 1.010  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 1, 2003-2012 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 0.999 | 1.178 | 1.199 | 1.221 | 1.083  |
| NS random walk          | 0.930 | 1.073 | 1.219 | 1.165 | 1.040  |
| NS-AR(1) rolling        | 1.004 | 1.181 | 1.193 | 1.192 | 1.078  |
| AR(1) on yields         | 1.041 | 1.028 | 1.042 | 1.021 | 1.014  |
| AR(1) on yields rolling | 1.043 | 1.019 | 1.037 | 1.020 | 1.007  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 1, 2013-2023 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.022 | 1.216 | 1.004 | 1.058 | 1.136  |
| NS random walk          | 0.989 | 1.209 | 0.999 | 1.043 | 1.147  |
| NS-AR(1) rolling        | 1.057 | 1.176 | 0.983 | 1.030 | 1.165  |
| AR(1) on yields         | 0.999 | 0.997 | 0.997 | 0.998 | 0.998  |
| AR(1) on yields rolling | 1.015 | 0.997 | 0.994 | 1.004 | 1.013  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 6, Full 2003-2023 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.234 | 1.285 | 1.187 | 1.171 | 1.119  |
| NS random walk          | 0.985 | 1.010 | 0.976 | 1.008 | 1.018  |
| NS-AR(1) rolling        | 1.270 | 1.273 | 1.129 | 1.111 | 1.111  |
| AR(1) on yields         | 1.179 | 1.120 | 1.166 | 1.104 | 1.081  |
| AR(1) on yields rolling | 1.137 | 1.080 | 1.130 | 1.090 | 1.070  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 6, 2003-2012 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.202 | 1.255 | 1.235 | 1.291 | 1.187  |
| NS random walk          | 0.979 | 1.000 | 0.986 | 1.040 | 0.990  |
| NS-AR(1) rolling        | 1.296 | 1.327 | 1.231 | 1.244 | 1.134  |
| AR(1) on yields         | 1.238 | 1.166 | 1.273 | 1.202 | 1.150  |
| AR(1) on yields rolling | 1.184 | 1.130 | 1.239 | 1.153 | 1.089  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 6, 2013-2023 (below 1 = more accurate than random walk)

|                  | y_3m  | y_18m | y_48m | y_90m | y_150m |
|------------------|-------|-------|-------|-------|--------|
| NS-AR(1)         | 1.311 | 1.373 | 1.099 | 1.006 | 1.015  |
| NS random walk   | 1.002 | 1.040 | 0.957 | 0.967 | 1.058  |
| NS-AR(1) rolling | 1.202 | 1.095 | 0.922 | 0.923 | 1.079  |

|                         |       |       |       |       |       |
|-------------------------|-------|-------|-------|-------|-------|
| AR(1) on yields         | 1.014 | 0.968 | 0.950 | 0.971 | 0.974 |
| AR(1) on yields rolling | 1.008 | 0.913 | 0.905 | 1.008 | 1.043 |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 |

RMSE relative to random walk, h = 12, Full 2003-2023 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.382 | 1.492 | 1.404 | 1.260 | 1.200  |
| NS random walk          | 0.990 | 1.011 | 0.968 | 1.002 | 1.015  |
| NS-AR(1) rolling        | 1.342 | 1.387 | 1.302 | 1.207 | 1.230  |
| AR(1) on yields         | 1.266 | 1.147 | 1.217 | 1.125 | 1.094  |
| AR(1) on yields rolling | 1.201 | 1.126 | 1.168 | 1.126 | 1.097  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 12, 2003-2012 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.273 | 1.419 | 1.526 | 1.558 | 1.404  |
| NS random walk          | 0.987 | 1.007 | 0.963 | 1.060 | 0.980  |
| NS-AR(1) rolling        | 1.381 | 1.485 | 1.555 | 1.537 | 1.374  |
| AR(1) on yields         | 1.350 | 1.215 | 1.396 | 1.305 | 1.209  |
| AR(1) on yields rolling | 1.284 | 1.219 | 1.357 | 1.247 | 1.151  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

RMSE relative to random walk, h = 12, 2013-2023 (below 1 = more accurate than random walk)

|                         | y_3m  | y_18m | y_48m | y_90m | y_150m |
|-------------------------|-------|-------|-------|-------|--------|
| NS-AR(1)                | 1.600 | 1.653 | 1.227 | 0.973 | 0.982  |
| NS random walk          | 0.998 | 1.020 | 0.974 | 0.957 | 1.045  |
| NS-AR(1) rolling        | 1.248 | 1.123 | 0.866 | 0.875 | 1.085  |
| AR(1) on yields         | 1.051 | 0.966 | 0.933 | 0.965 | 0.979  |
| AR(1) on yields rolling | 0.988 | 0.865 | 0.862 | 1.025 | 1.047  |
| Random walk             | 1.000 | 1.000 | 1.000 | 1.000 | 1.000  |

```
In [23]: # Empty list that will hold one row per test
rows = []
# Loop over each diagnostic model
for m in NEW_MODELS:
    # Loop over each horizon
    for h in HORIZONS:
        # Start a new row, labelled with the horizon and the comparison
        row = {"Test": f"h={h}: {m} vs Random walk"}
        # Loop over each maturity
        for col in MAIN_COLS:
            # Run the Diebold-Mariano test of this model against the random walk
            dm, p = diebold_mariano(errors[h][m][col], errors[h]["Random walk"][col])
            # Store the statistic with its significance marker
            row[col] = f"{dm:.2f}" + ("**" if p < 0.05 else ("*" if p < 0.10 else ""))
        # Add this row to the list
        rows.append(row)

# Print a title for the table
print("Table 8: Diebold-Mariano tests, diagnostic models against the random walk")
# Print an explanation of the sign convention
print("Negative values: diagnostic model more accurate. ** significant at 5%, * sig
```

```
# Print the table
print(pd.DataFrame(rows).set_index("Test").to_string())
```

Table 8: Diebold-Mariano tests, diagnostic models against the random walk

Negative values: diagnostic model more accurate. \*\* significant at 5%, \* significant at 10%.

|                                              | y_3m    | y_18m  | y_48m  | y_90m  | y_150 |
|----------------------------------------------|---------|--------|--------|--------|-------|
| m                                            |         |        |        |        |       |
| Test                                         |         |        |        |        |       |
| h=1: NS random walk vs Random walk           | -2.19** | 2.46** | 2.40** | 2.60** | 3.10* |
| *                                            |         |        |        |        |       |
| h=6: NS random walk vs Random walk           | -1.38   | 0.36   | -0.87  | 0.30   | 0.8   |
| 2                                            |         |        |        |        |       |
| h=12: NS random walk vs Random walk          | -1.35   | 0.54   | -1.53  | 0.10   | 0.6   |
| 7                                            |         |        |        |        |       |
| h=1: NS-AR(1) rolling vs Random walk         | 0.86    | 3.53** | 2.11** | 2.76** | 3.50* |
| *                                            |         |        |        |        |       |
| h=6: NS-AR(1) rolling vs Random walk         | 3.27**  | 2.41** | 1.45   | 1.66*  | 1.6   |
| 2                                            |         |        |        |        |       |
| h=12: NS-AR(1) rolling vs Random walk        | 1.69*   | 1.64   | 1.44   | 1.40   | 1.65  |
| *                                            |         |        |        |        |       |
| h=1: AR(1) on yields rolling vs Random walk  | 3.14**  | 1.17   | 1.28   | 0.86   | 1.0   |
| 0                                            |         |        |        |        |       |
| h=6: AR(1) on yields rolling vs Random walk  | 2.08**  | 1.12   | 1.41   | 1.25   | 1.3   |
| 0                                            |         |        |        |        |       |
| h=12: AR(1) on yields rolling vs Random walk | 1.41    | 1.01   | 1.21   | 1.21   | 1.3   |
| 6                                            |         |        |        |        |       |

## Step 11: Robustness checks

**A. Long-band maturity assumption.** The "10 years and over" band is re-assigned from 150 to 180 months, and the factors, fit and NS-AR(1) forecasts are re-estimated.

**B. All available maturities.** Each month is fitted using every available maturity (between 5 and 9), from January 1994 to May 2026, including the months with no 3 to 5 year yield.

```
In [24]: # Define a function that produces NS-AR(1) forecast errors for a given set of factors
# (the same recursive, direct method as Step 9)
def ns_ar1_errors(fvals, Xload, h):
    # Empty lists for the forecasts and their target dates
    fc_list, targets = [], []
    # Loop over each forecast date, as in Step 9
    for i in range(i0, len(yields) - h):
        # Factor pairs available at date i: today (Zf) and h months later (Yf)
        Zf, Yf = fvals[:, i - h + 1], fvals[:, i + 1]
        # Forecast each factor separately from its own current value
        b_fc = np.array([direct_forecast(Yf[:, j], Zf[:, [j]], fvals[i, j]) for j in range(1, N)])
        # Turn the forecast factors into forecast yields
        fc_list.append(Xload @ b_fc)
        # Record the date being forecast
        targets.append(yields.index[i + h])
    # Put the forecasts in a table with target dates as rows and maturities as columns
    fc = pd.DataFrame(np.vstack(fc_list), index=targets, columns=MAIN_COLS)
    # Return the forecast errors: actual minus forecast
```

```

    return yields.loc[targets] - fc

# Copy the main maturities so the original stays unchanged
tau_180 = tau.copy()
# Replace the last maturity (the "10 years and over" band) with 180 months
tau_180[-1] = 180
# Nelson-Siegel loadings for the alternative maturities
X180 = ns_loadings(tau_180, LAMBDA)
# Re-estimate the three factors for every month by OLS, using the alternative loadings
coef180 = np.linalg.lstsq(X180, yields.values.T, rcond=None)[0]
# Put the alternative factors in a table
factors180 = pd.DataFrame(coef180.T, index=yields.index, columns=["beta1", "beta2", "beta3"])
# Residuals of the alternative fit: actual minus fitted yields
resid180 = yields - pd.DataFrame(factors180.values @ X180.T, index=yields.index, columns=yields.columns)

# Build a table comparing the factors under the two assumptions
comp = pd.DataFrame({
    "Mean (150m)": factors.mean(),
    "Mean (180m)": factors180.mean(),
    "Std (150m)": factors.std(),
    "Std (180m)": factors180.std(),
    "Correlation": [factors[c].corr(factors180[c]) for c in factors.columns],
})
# Print a title
print("Robustness A: factors with the long band at 150 months (main) and 180 months")
# Print the comparison rounded to 3 decimal places
print(comp.round(3).to_string())

# Build a table comparing the in-sample fit (residual RMSE) under the two assumptions
fit = pd.DataFrame({"RMSE (150m)": np.sqrt((resid ** 2).mean()), "RMSE (180m)": np.sqrt((resid180 ** 2).mean())})
# Print a title
print("\nRobustness A: in-sample residual RMSE by maturity")
# Print the fit comparison
print(fit.round(3).to_string())

# Empty list that will hold RMSE ratios for each horizon and assumption
rows = []
# Loop over each forecast horizon
for h in HORIZONS:
    # NS-AR(1) forecast errors under the 180-month assumption
    e180 = ns_ar1_errors(factors180.values, X180, h)
    # Random walk RMSE at each maturity (unchanged, since the yields themselves are unchanged)
    rmse_rw = np.sqrt((errors[h]["Random walk"] ** 2).mean())
    # NS-AR(1) RMSE relative to random walk, main assumption
    rows.append(pd.Series(np.sqrt((errors[h]["NS-AR(1)"] ** 2).mean()) / rmse_rw, name=f"h={h}, 150m"))
    # NS-AR(1) RMSE relative to random walk, alternative assumption
    rows.append(pd.Series(np.sqrt((e180 ** 2).mean()) / rmse_rw, name=f"h={h}, 180m"))
# Print a title
print("\nRobustness A: NS-AR(1) RMSE relative to random walk (below 1 = more accurate)")
# Print the comparison
print(pd.DataFrame(rows).round(3).to_string())

```

Robustness A: factors with the long band at 150 months (main) and 180 months

|       | Mean (150m) | Mean (180m) | Std (150m) | Std (180m) | Correlation |
|-------|-------------|-------------|------------|------------|-------------|
| beta1 | 10.925      | 10.854      | 2.777      | 2.772      | 0.999       |
| beta2 | -2.057      | -2.001      | 2.802      | 2.750      | 1.000       |
| beta3 | -1.923      | -1.739      | 3.410      | 3.190      | 0.999       |

Robustness A: in-sample residual RMSE by maturity

|        | RMSE (150m) | RMSE (180m) |
|--------|-------------|-------------|
| y_3m   | 0.064       | 0.068       |
| y_18m  | 0.189       | 0.200       |
| y_48m  | 0.198       | 0.195       |
| y_90m  | 0.149       | 0.152       |
| y_150m | 0.133       | 0.129       |

Robustness A: NS-AR(1) RMSE relative to random walk (below 1 = more accurate than random walk)

|            | y_3m  | y_18m | y_48m | y_90m | y_150m |
|------------|-------|-------|-------|-------|--------|
| h=1, 150m  | 1.007 | 1.190 | 1.098 | 1.129 | 1.113  |
| h=1, 180m  | 0.993 | 1.209 | 1.094 | 1.113 | 1.111  |
| h=6, 150m  | 1.234 | 1.285 | 1.187 | 1.171 | 1.119  |
| h=6, 180m  | 1.218 | 1.277 | 1.167 | 1.145 | 1.110  |
| h=12, 150m | 1.382 | 1.492 | 1.404 | 1.260 | 1.200  |
| h=12, 180m | 1.357 | 1.465 | 1.358 | 1.213 | 1.161  |

```
In [25]: # Empty lists for each month's estimated factors and the number of maturities used
fac_rows, n_used = [], []
# Loop over every month in the full dataset (January 1994 to May 2026)
for date, row in yields_all.iterrows():
    # Keep only the maturities that have a yield this month
    avail = row.dropna()
    # Nelson-Siegel loadings for this month's available maturities
    Xa = ns_loadings([MATURITY[c] for c in avail.index], LAMBDA)
    # Estimate this month's three factors by OLS and store them
    fac_rows.append(np.linalg.lstsq(Xa, avail.values, rcond=None)[0])
    # Store how many maturities were used this month
    n_used.append(len(avail))

# Put the factors in a table with one row per month
factors_all = pd.DataFrame(fac_rows, index=yields_all.index, columns=["beta1", "beta2", "beta3"])
# Save these factors to the Output folder
factors_all.to_csv("../Output/ns_factors_all_maturities.csv")
# Nelson-Siegel loadings for all nine maturities
X_all = ns_loadings([MATURITY[c] for c in ALL_COLS], LAMBDA)
# Fitted yields at all nine maturities, for every month
fitted_all = pd.DataFrame(factors_all.values @ X_all.T, index=yields_all.index, columns=ALL_COLS)
# Residuals: actual minus fitted (missing where no actual yield exists)
resid_all = yields_all - fitted_all

# Print a title
print("Robustness B: number of months by number of maturities used")
# Count how many months used 5, 6, 7, 8 and 9 maturities
print(pd.Series(n_used, index=yields_all.index).value_counts().sort_index().to_string())

# Build a table comparing residual RMSE: all-maturity fit against the main 5-maturity fit
fit_all = pd.DataFrame({
    "Months": resid_all.count(),
    #
```

```

    "RMSE (all maturities)": np.sqrt((resid_all ** 2).mean()), #
    "RMSE (main, 5 maturities)": np.sqrt((resid ** 2).mean()).reindex(ALL_COLS), #
})
# Print a title
print("\nRobustness B: in-sample residual RMSE by maturity")
# Print the comparison
print(fit_all.round(3).to_string())

# Print a title
print("\nRobustness B: correlation with main factors, 1994:01 to 2023:02")
# Correlation between main and all-maturity factors over the months they share
print(pd.Series({c: factors[c].corr(factors_all.loc[factors.index, c]) for c in fac

# Print a title
print("\nRobustness B: factor statistics after the main sample, 2023:03 to 2026:05")
# Summary statistics of the all-maturity factors in the months beyond the main samp
print(factors_all.loc["2023-03-31:].describe().round(3).to_string())

# Create three charts stacked vertically, sharing the same time axis
fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=True)
# Loop over each chart, factor and label together
for ax, c, lab in zip(axes, factors.columns, ["β1 (level)", "β2 (slope)", "β3 (curv
    # Main-specification factor as a solid line (ends February 2023)
    ax.plot(factors.index, factors[c], color="black", linewidth=1.2, label=lab + ",
    # All-maturity factor as a dotted line (runs to May 2026)
    ax.plot(factors_all.index, factors_all[c], color="black", linewidth=1.0, linest
    # Dashed vertical line marking the end of the main sample
    ax.axvline(pd.Timestamp("2023-02-28"), color="black", linewidth=0.8, linestyle=
    # Label the vertical axis
    ax.set_ylabel("Percent")
    # Add a legend without a box, placed where it overlaps least
    ax.legend(frameon=False, loc="best")
    # Add light grey grid lines
    ax.grid(color="lightgrey", linewidth=0.5)
    # Remove the top and right borders
    ax.spines[["top", "right"]].set_visible(False)

# Label the shared time axis on the bottom chart
axes[2].set_xlabel("Date")
# Adjust spacing so labels do not overlap
fig.tight_layout()
# Save the figure to the Output folder at print quality
fig.savefig("../Output/fig8_factors_all_maturities.png", dpi=300, bbox_inches="tigh
# Show the figure in the notebook
plt.show()

```

Robustness B: number of months by number of maturities used

|   |     |
|---|-----|
| 5 | 96  |
| 6 | 61  |
| 7 | 40  |
| 8 | 102 |
| 9 | 90  |

Robustness B: in-sample residual RMSE by maturity

|        | Months | RMSE (all maturities) | RMSE (main, 5 maturities) |
|--------|--------|-----------------------|---------------------------|
| y_3m   | 389    | 0.255                 | 0.064                     |
| y_6m   | 293    | 0.148                 | NaN                       |
| y_9m   | 232    | 0.256                 | NaN                       |
| y_12m  | 192    | 0.278                 | NaN                       |
| y_18m  | 389    | 0.248                 | 0.189                     |
| y_48m  | 366    | 0.220                 | 0.198                     |
| y_90m  | 389    | 0.233                 | 0.149                     |
| y_150m | 389    | 0.134                 | 0.133                     |
| y_300m | 113    | 0.234                 | NaN                       |

Robustness B: correlation with main factors, 1994:01 to 2023:02

|       |       |
|-------|-------|
| beta1 | 0.999 |
| beta2 | 0.995 |
| beta3 | 0.992 |

Robustness B: factor statistics after the main sample, 2023:03 to 2026:05

|       | beta1  | beta2  | beta3  |
|-------|--------|--------|--------|
| count | 39.000 | 39.000 | 39.000 |
| mean  | 11.534 | -3.188 | -4.877 |
| std   | 1.204  | 0.759  | 1.413  |
| min   | 8.881  | -4.659 | -6.871 |
| 25%   | 11.076 | -3.798 | -5.776 |
| 50%   | 11.805 | -3.243 | -5.254 |
| 75%   | 12.491 | -2.553 | -3.935 |
| max   | 13.136 | -1.797 | -1.565 |

